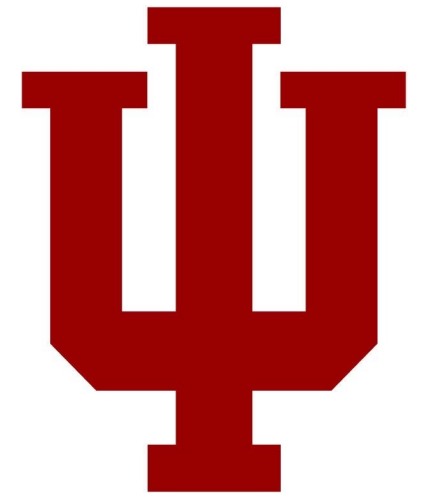


03:

~~01~~: Special Signals

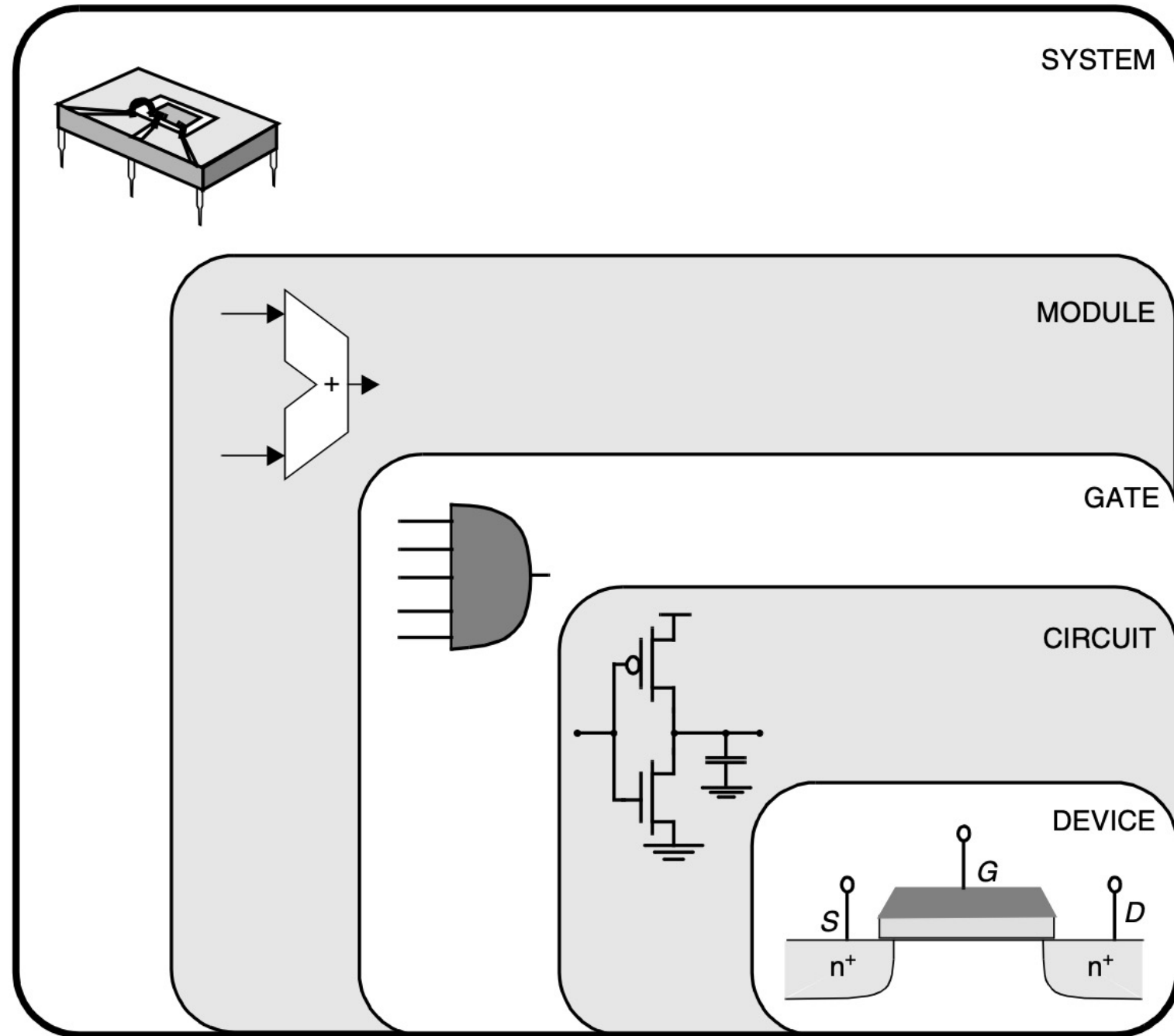
E599– Digital Integrated Circuit Design



Website

<https://engr599-ic.github.io>

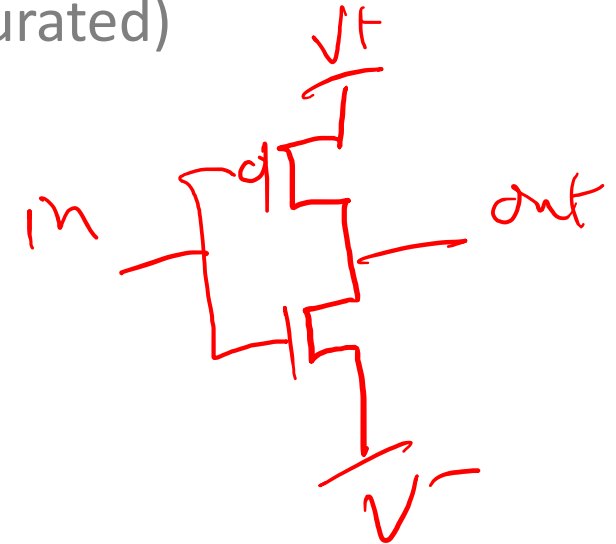
Review Time



Devices Review

- Transistors

- Voltage on gate causes conduction between source + drain. Aka “Magic”
- Only consider fully-off or fully-on (saturated)
- Two types: NMOS and PMOS
 - PMOS: 0V=on
 - NMOS: 0V =off



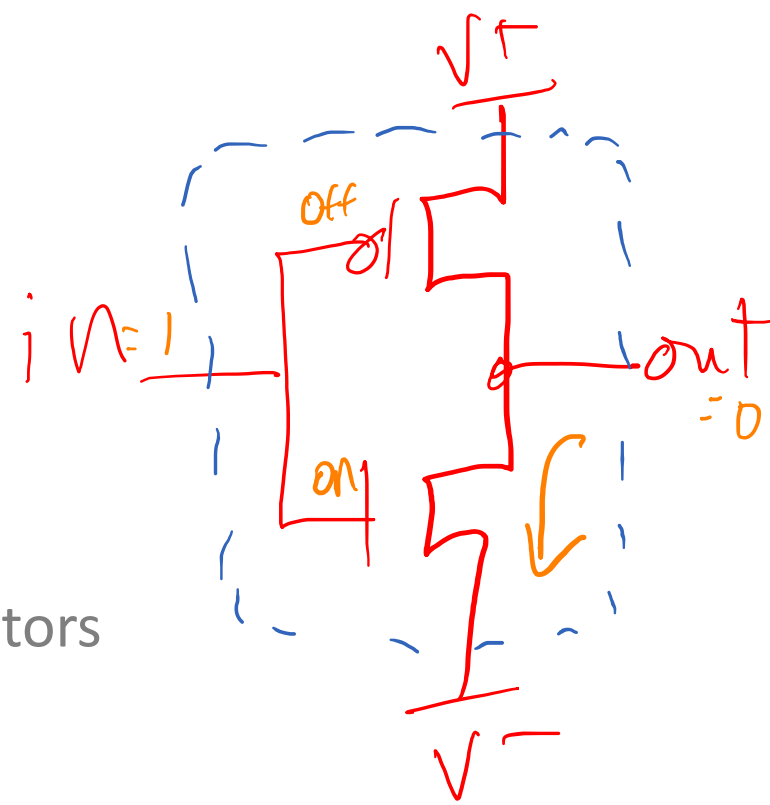
- Wires

- Resistor + Capacitor
- Longer wires = slower signals

Combinational Circuits Review

- Inverter: 1 NMOS + 1 PMOS
- Process Variation:
 - Some faster, some slower transistors
- Intrinsic Delay:
 - Fundamental delay in all transitions
 - Bigger transistors reduce delays, to a point.
- Other Delays
 - output takes times to reflect input changes
 - Bigger capacitive loads increase delays

fanout:



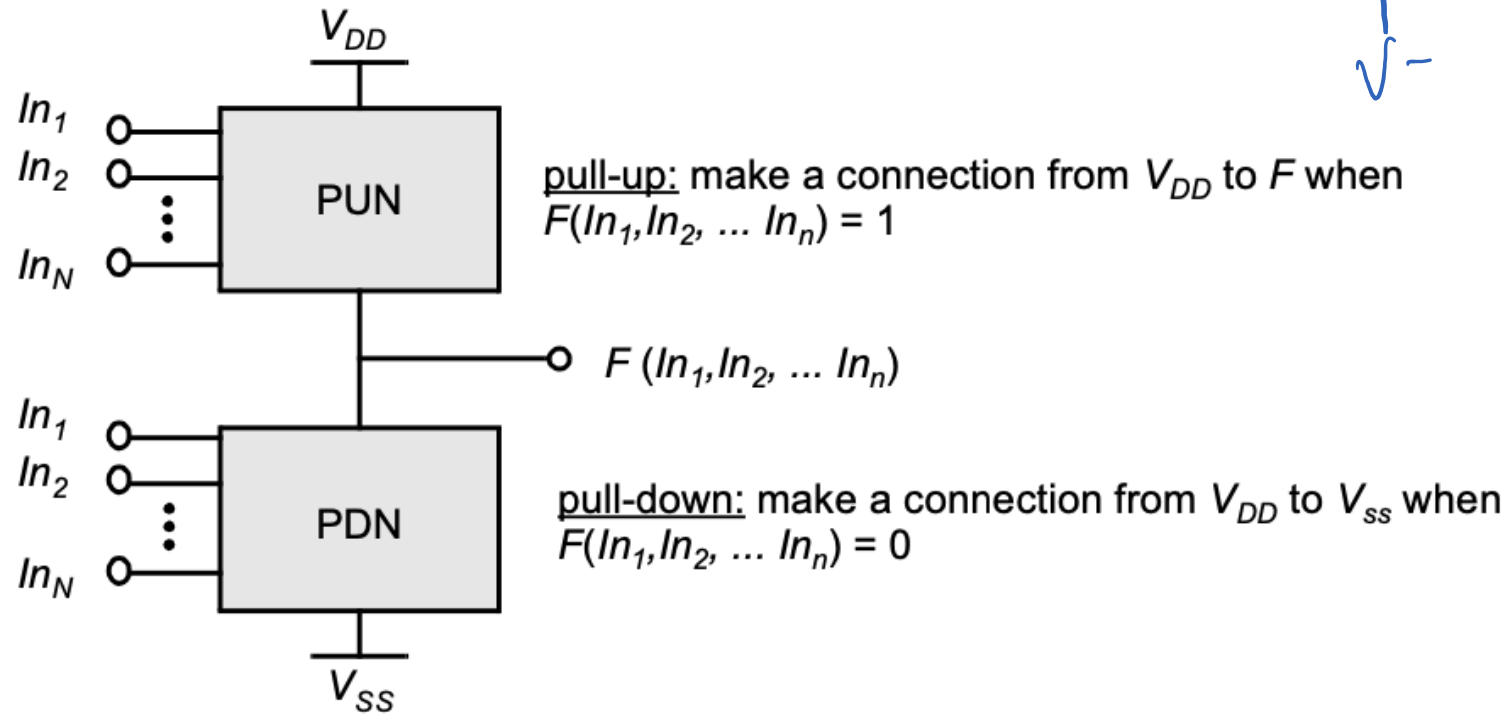
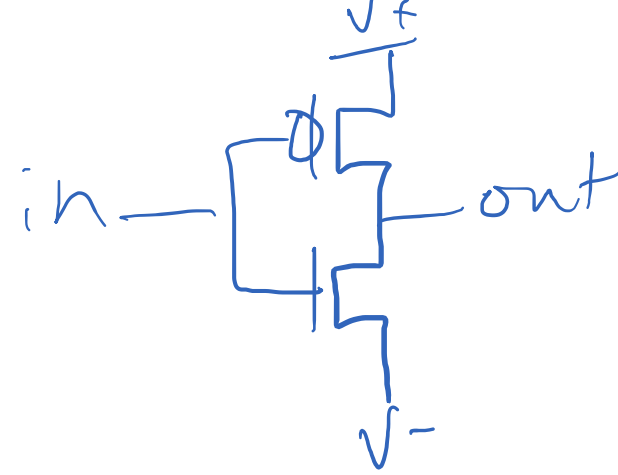
Sequential Circuits Review

- Latches: Output follows input when clock is high
- Flip-Flop: Output follows input on the rising edge of the clock
- Setup Time: Time input must be stable before the clock rising edge
- Hold Time: Time input must be stable after the clock rising edge
- Setup affects performance. **Hold affects correctness.**

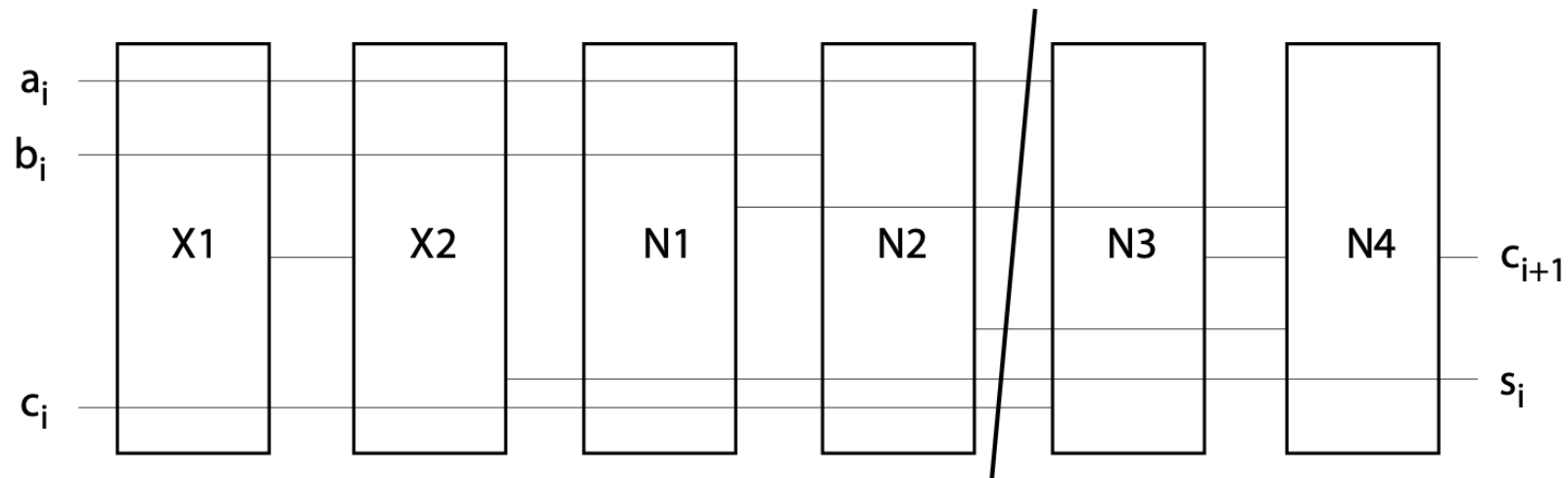
WARNING: HOLD TIMES ARE A BIG DEAL!!!

- You can always fix setup times with a slower clock.
- Hold times are unfixable....
- You do not want a hold-time violation in your design!

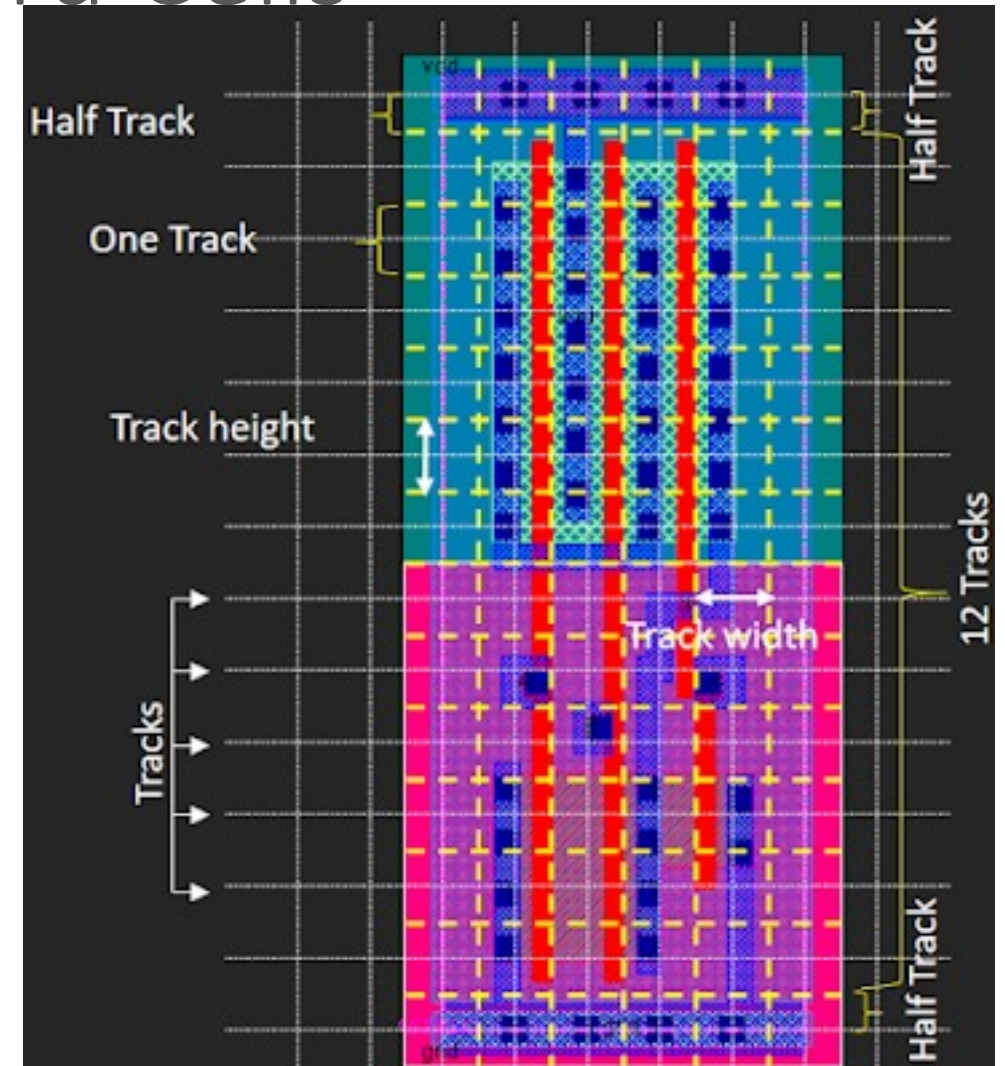
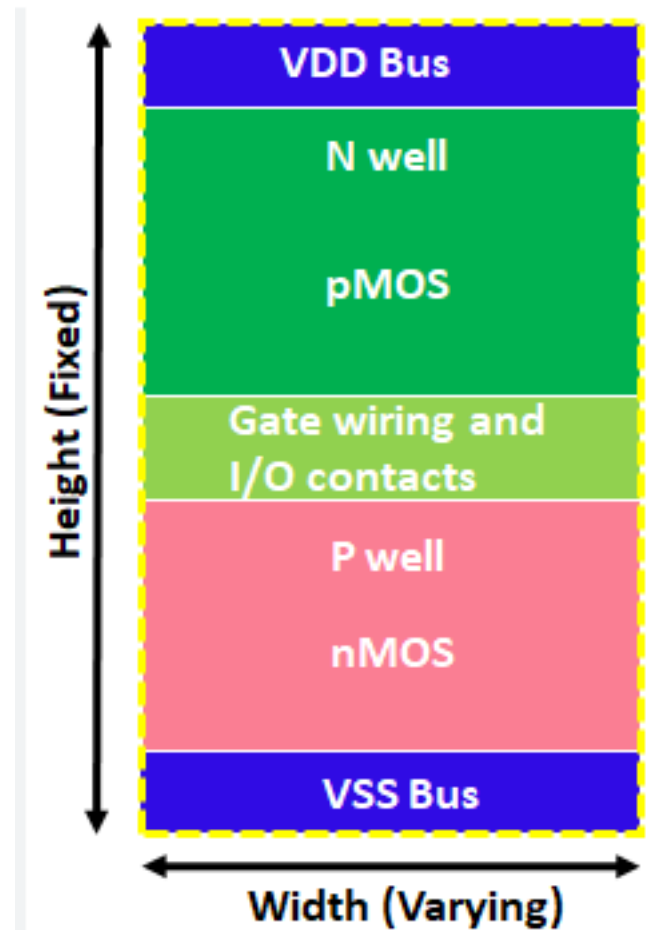
Standardized Logic Gates



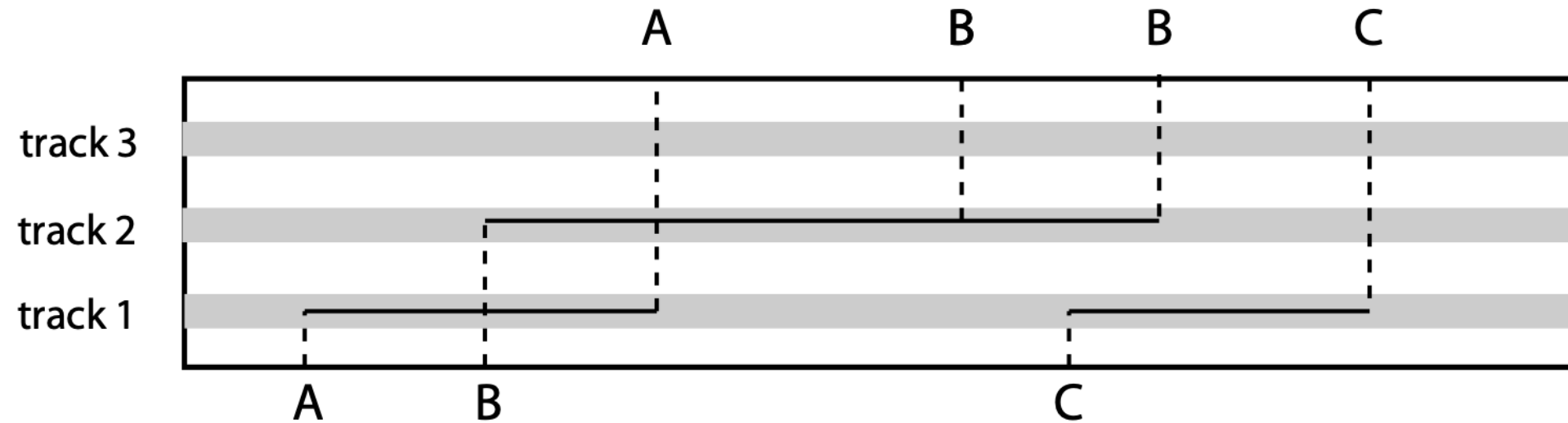
Put the wires above the logic cells



Routing Tracks w/in Standard Cells



Routing Tracks



The “CLK” (Clock)

- In digital logic, a clock signal is a periodic electrical signal, typically a square wave, that oscillates between high and low states to provide a precise timing reference for synchronous circuits [wiki]

Getting the CLK

- Where does CLK come from?

— CLK input

RC circuits

• Oscillator
crystal

"voltage controlled
oscillator"

"phase locked loop"

"Delay Locked Loop"

Getting the CLK

- Where does CLK come from?

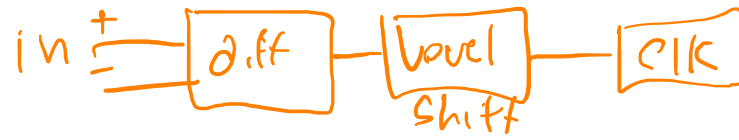
- Outside world

- Limited frequency to $<200\text{MHz}$



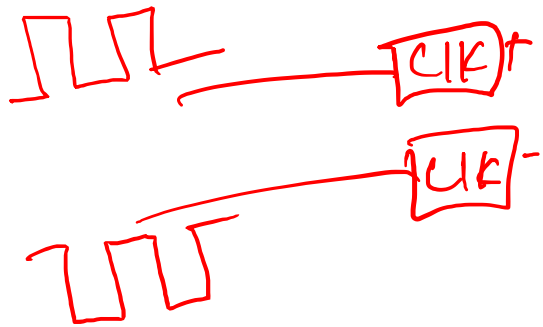
- Inside IC

- Requires additional hardware
 - Often requires outside reference clock

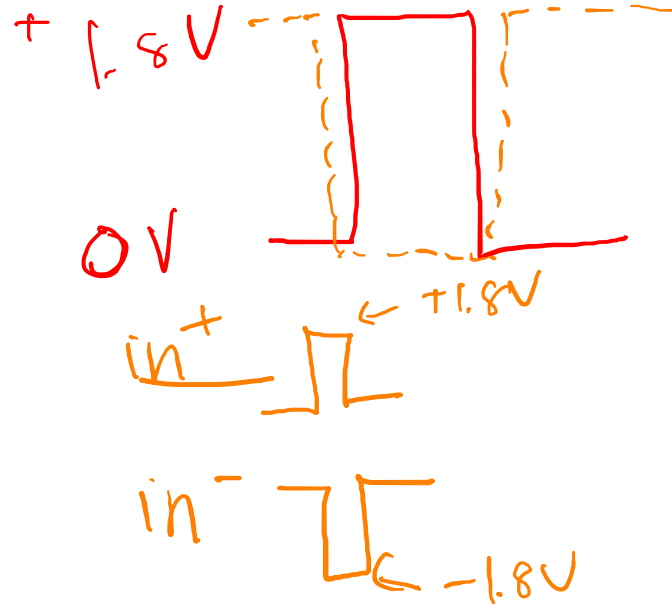


CLKs from the outside world

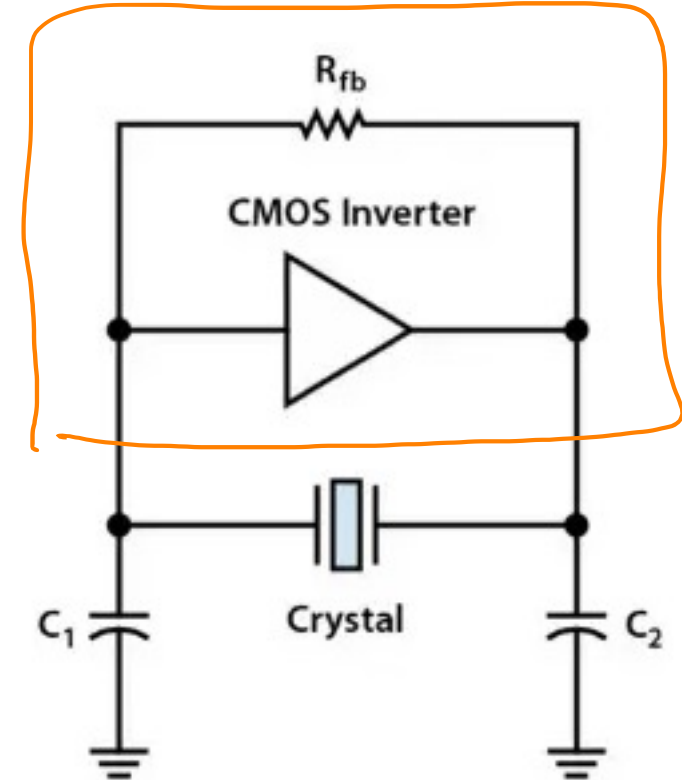
Magic CLK



- Single-ended signal
- Differential signal
- Crystal Oscillator



on chip



CLKs from inside the IC

- Need a block to generate >200MHz on-chip CLK

- Lots of options
 - Ring Oscillator
 - RC (LC) oscillator



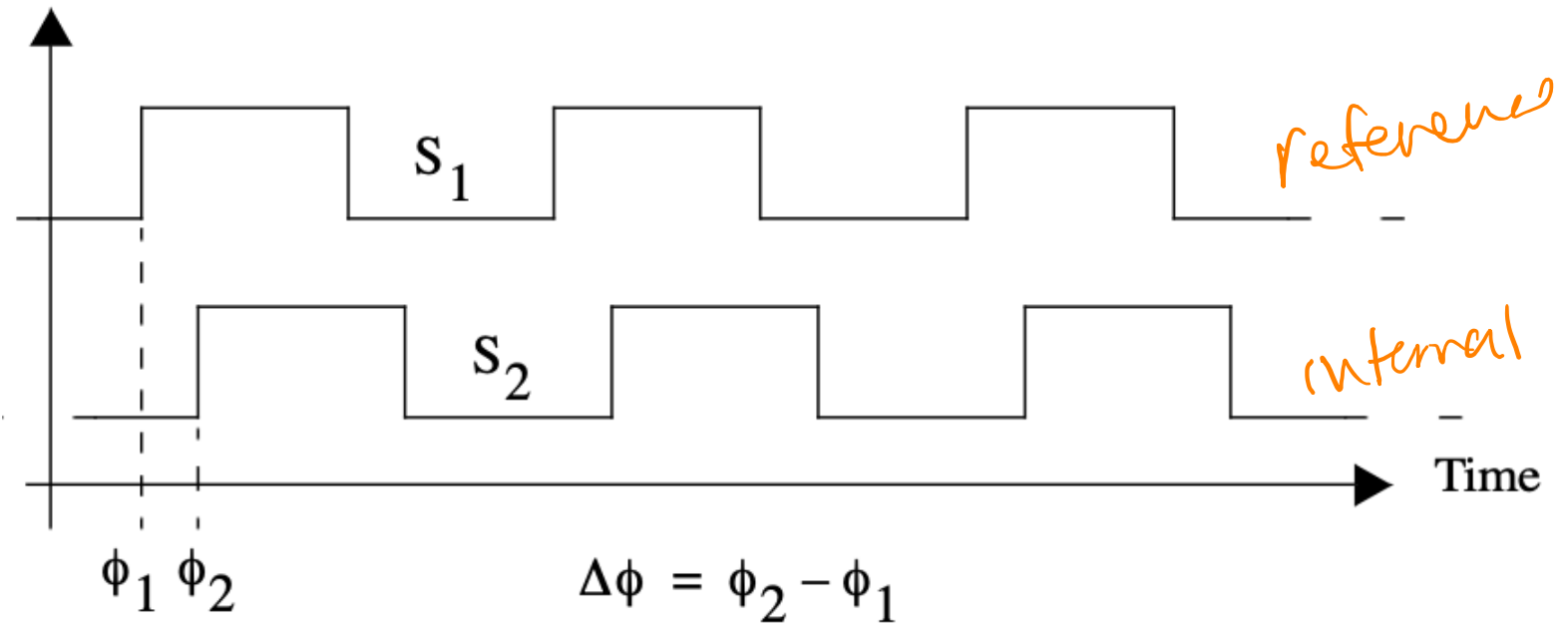
- **Phase-Locked Loop (PLL)**
- Delay-Locked Lopp (DLL)

regain
reference
freq.
CLK
slower

PLLs

- Basics:
 - Input: Low-frequency off-chip “reference” clock
 - Magic
 - Output: high-frequency on-chip clock

PLL Basics



PLL Basics

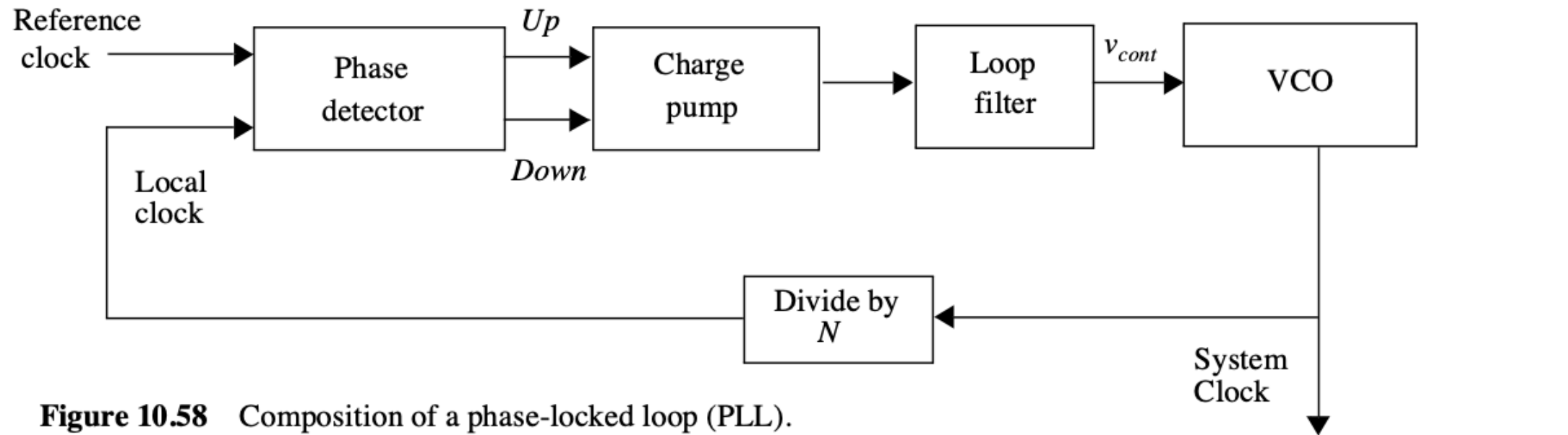
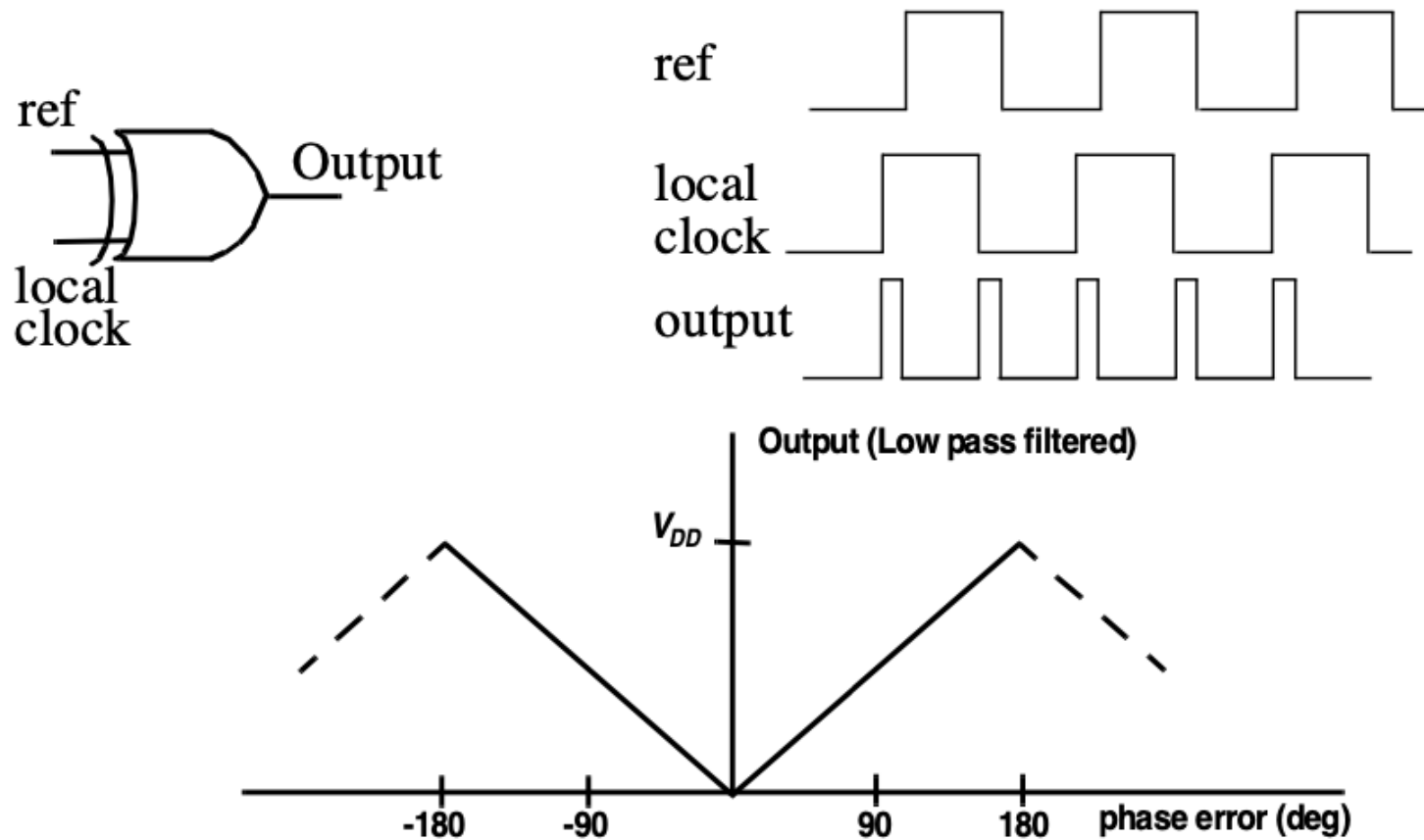


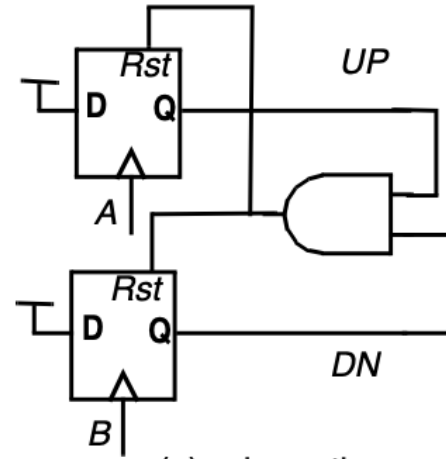
Figure 10.58 Composition of a phase-locked loop (PLL).

PLL Basics

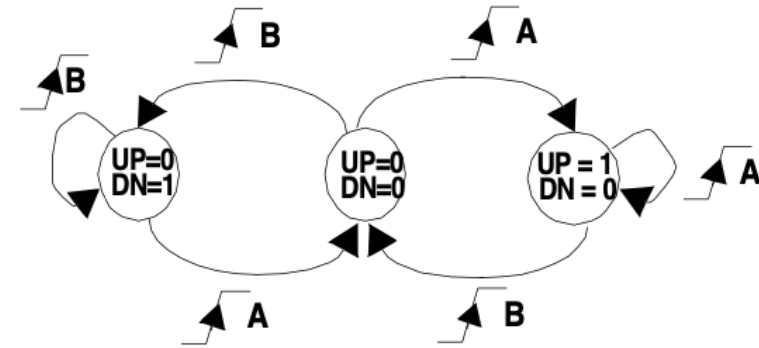


Can lock onto the wrong multiple of clock frequency

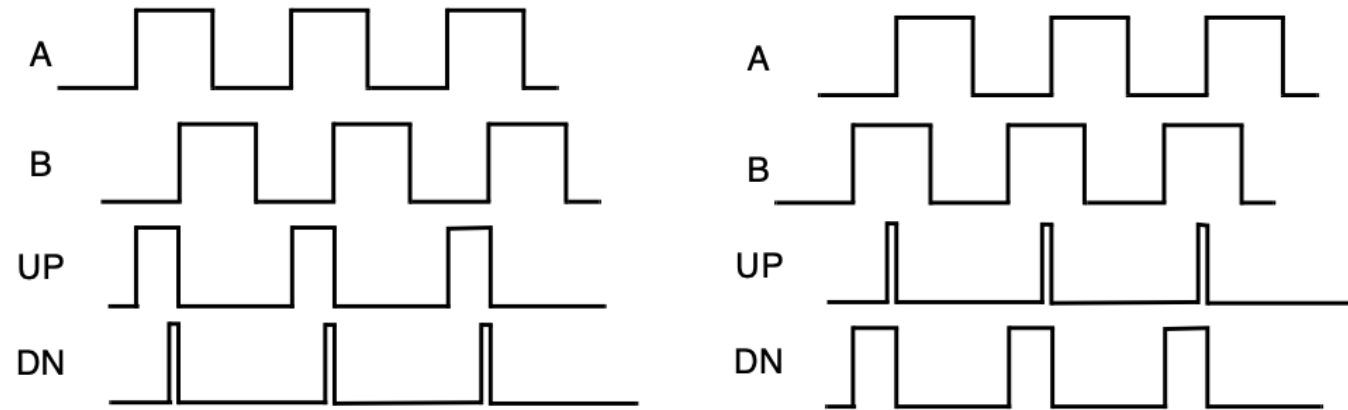
PLL Basics



(a) schematic



(b) state transition diagram



(c) Timing waveforms

PLL Basics

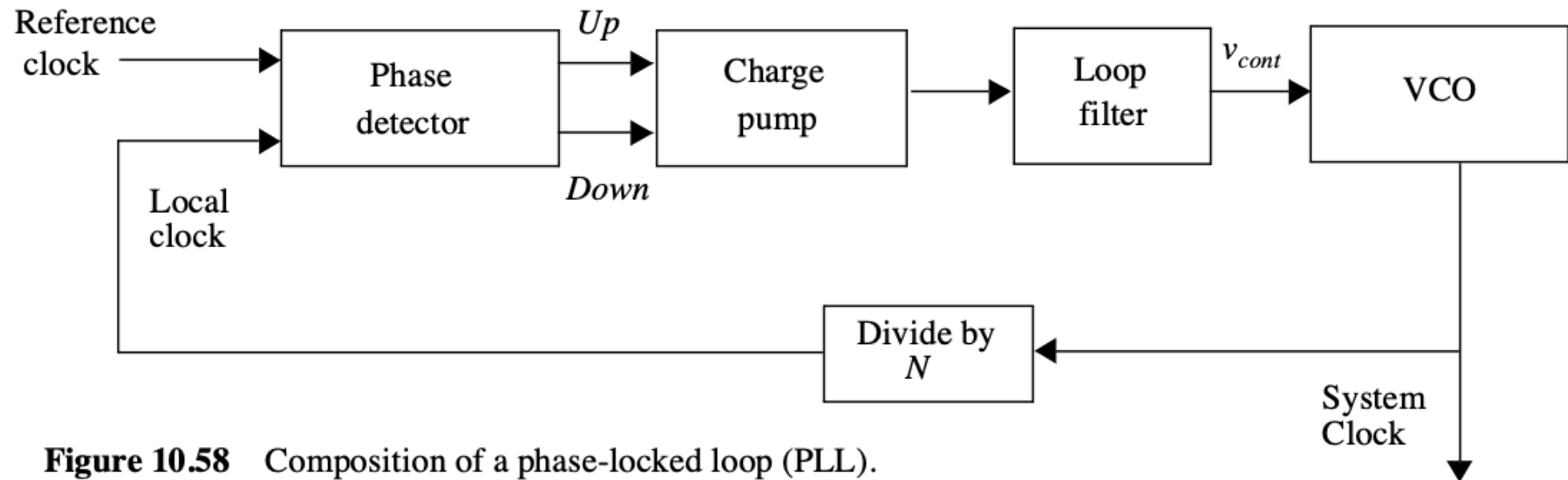
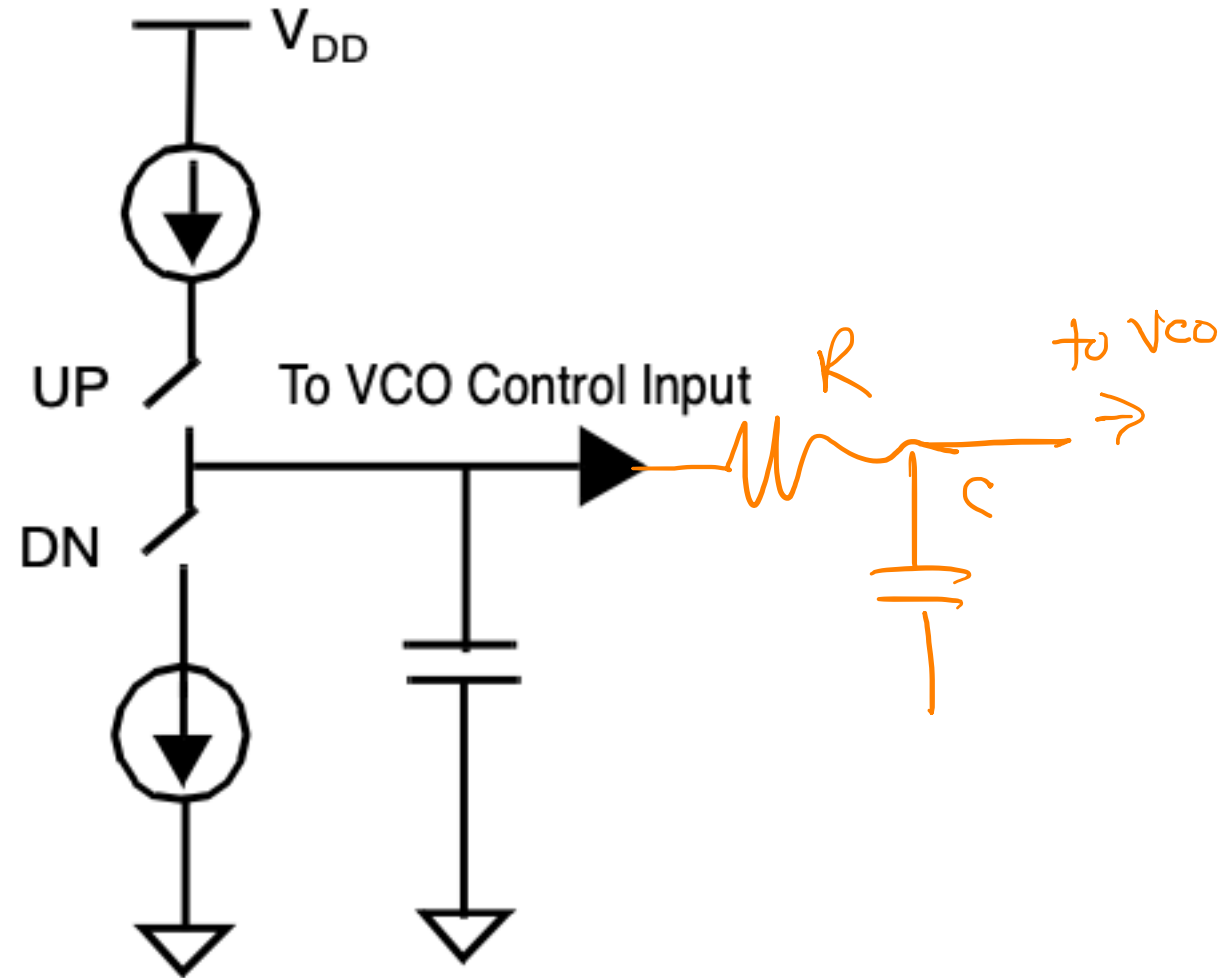


Figure 10.58 Composition of a phase-locked loop (PLL).

PLL Basics



PLL Basics

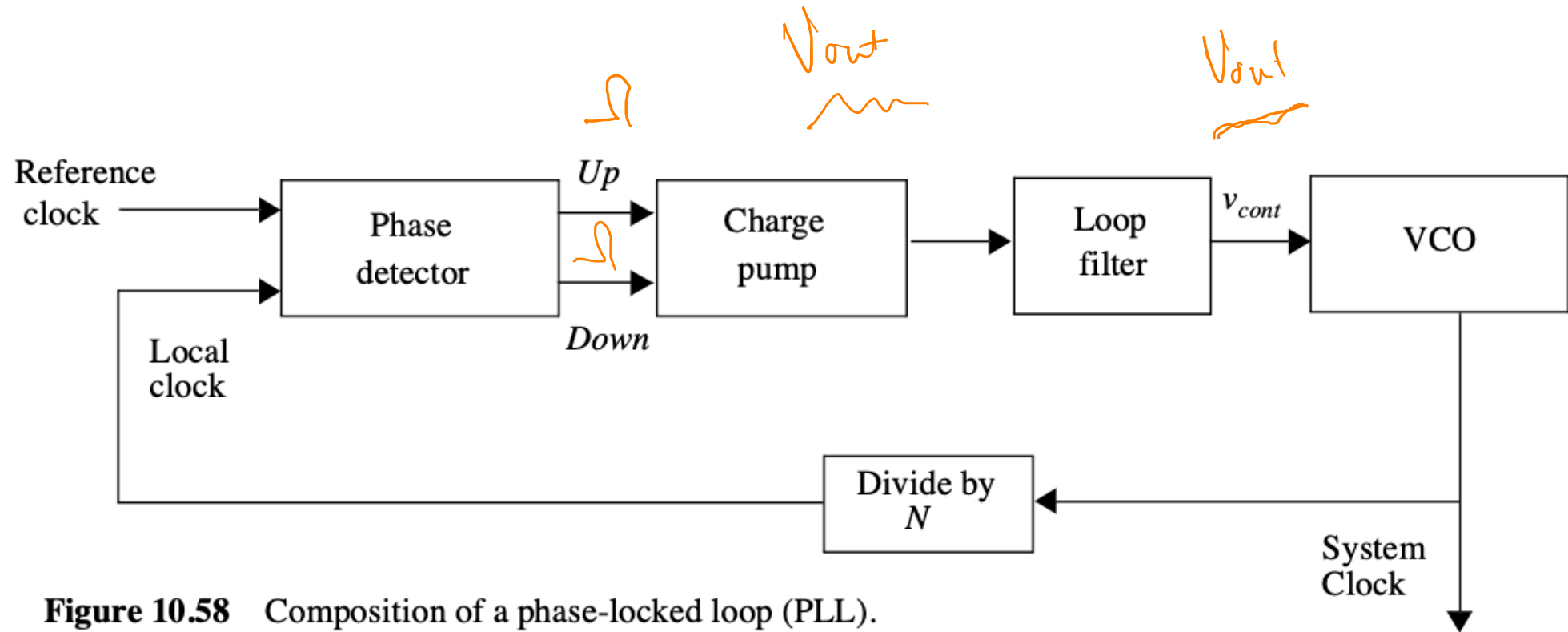
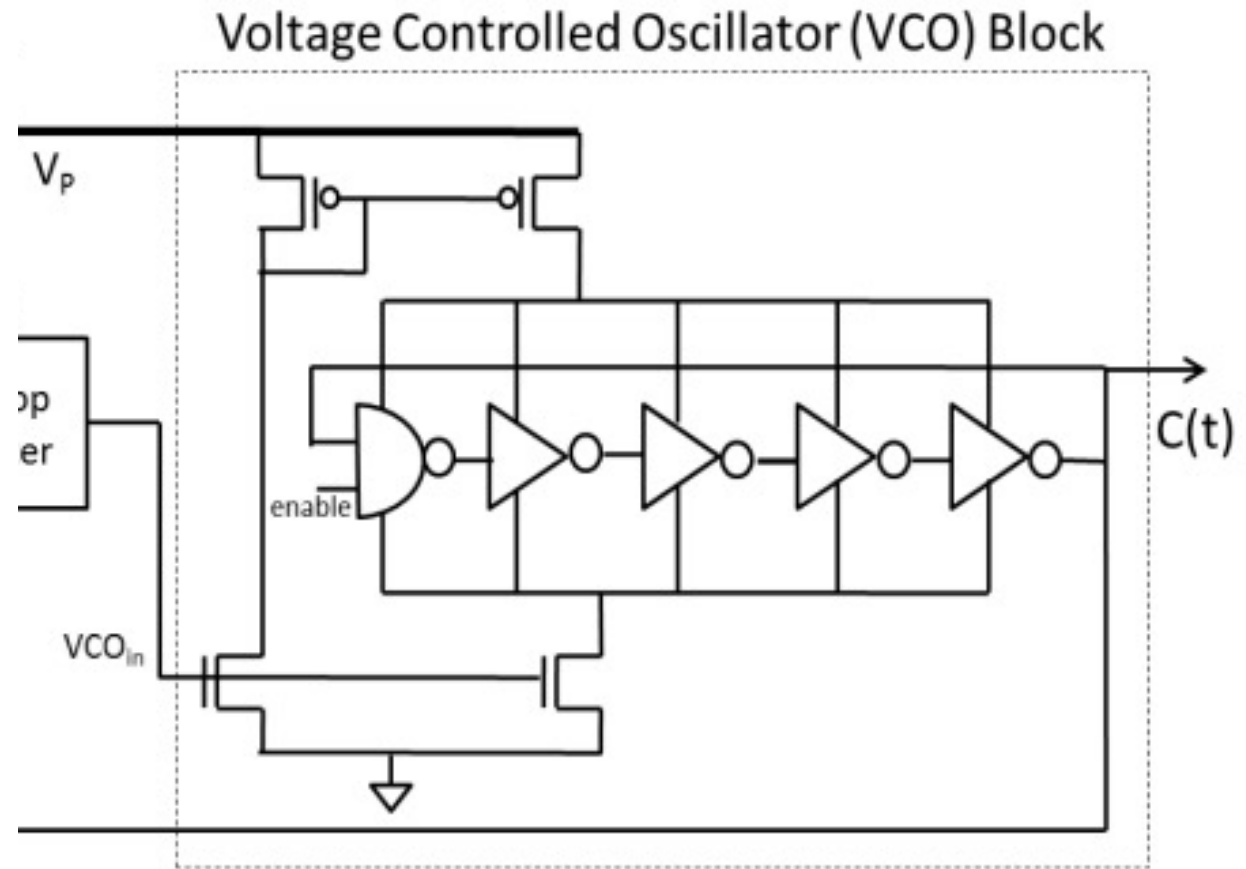


Figure 10.58 Composition of a phase-locked loop (PLL).

PLL Basics



PLL Basics

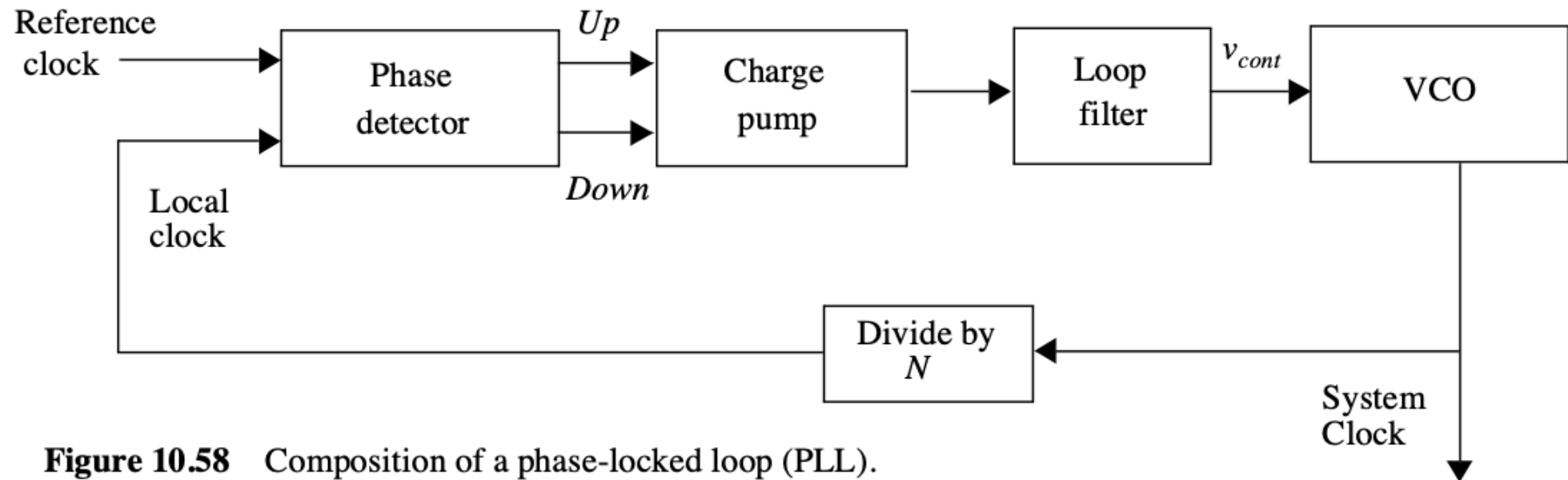
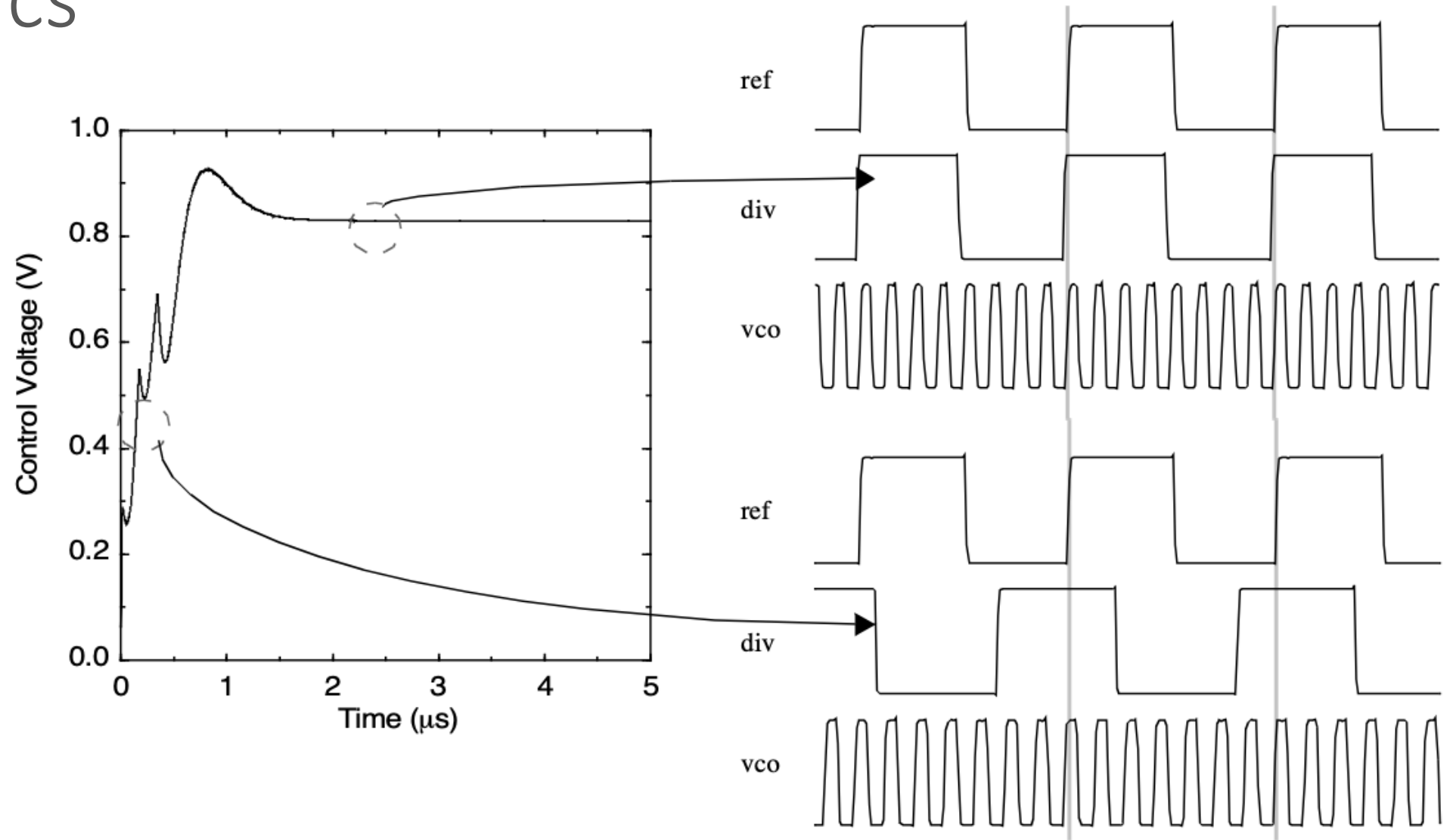
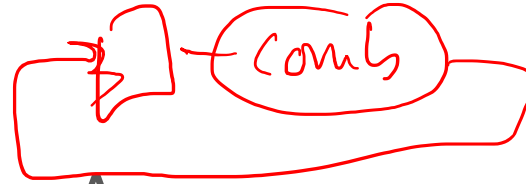


Figure 10.58 Composition of a phase-locked loop (PLL).

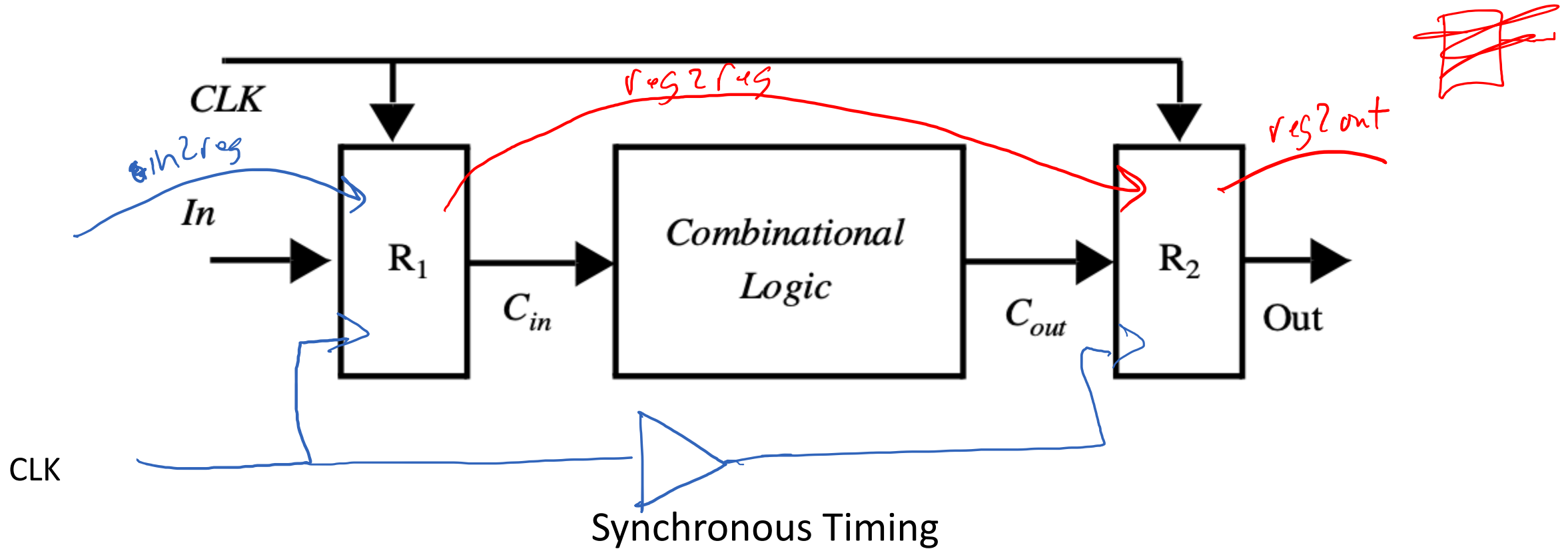
PLL Basics



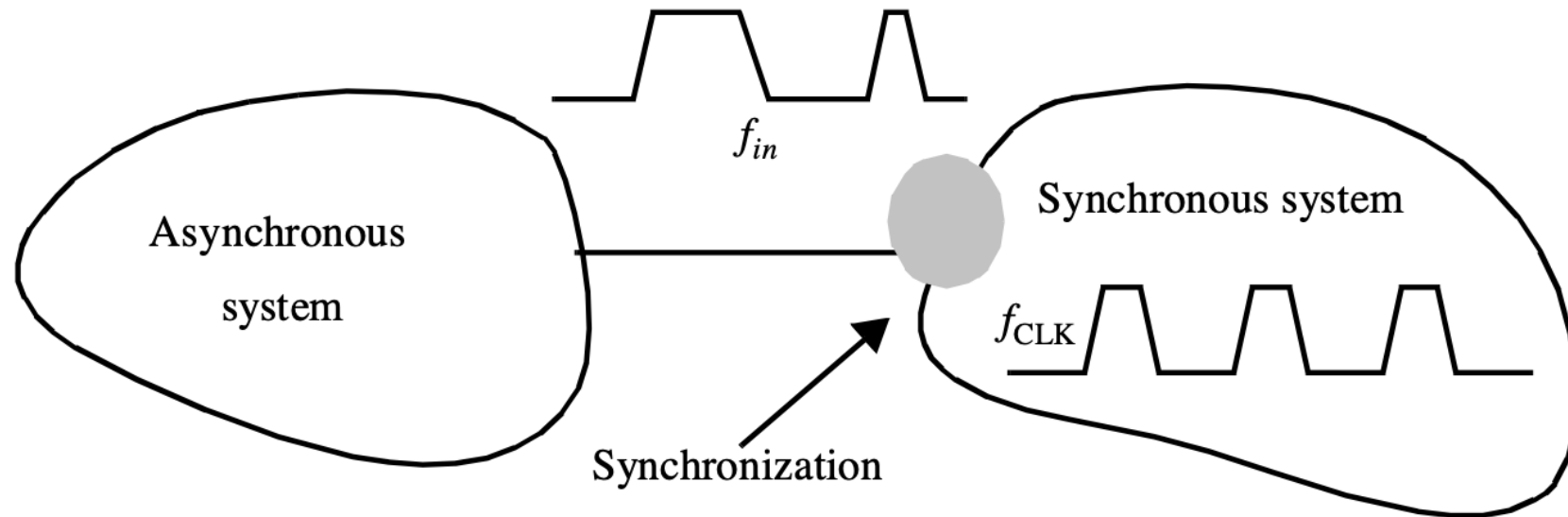
Different Timing Arcs



Reg2reg
In2reg
reg2out



How do we deal with asynchronous inputs?



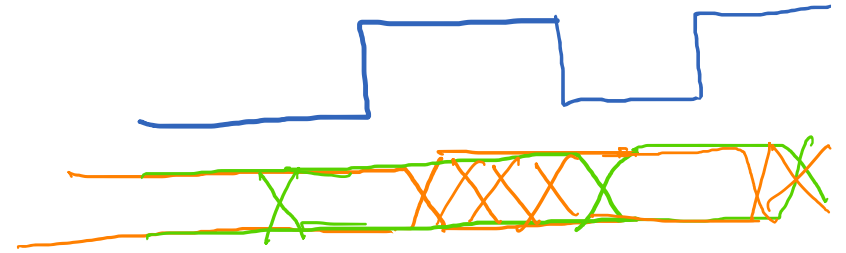
We force them to be synchronous

- Use SDCs to define input/output timing constraints
- Works well for blocks.

```
set_input_delay -clock <clock_name>  
[-max] [-min] <delay_value>  
[get_ports <port_list>]
```

clk

in0



```
create_clock -name clk -period 500 [get_ports {clk}]
```

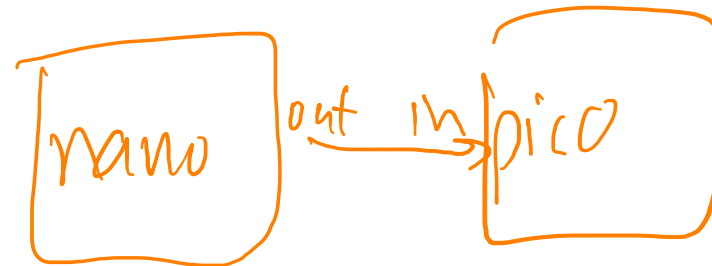
100ps

```
set_input_delay -clock [get_clocks clk] -min -add_delay 100 [all_inputs]
```

```
set_input_delay -clock [get_clocks clk] -max -add_delay 400 [all_inputs]
```

```
set_output_delay -clock [get_clocks clk] -min -add_delay 2 [all_outputs]
```

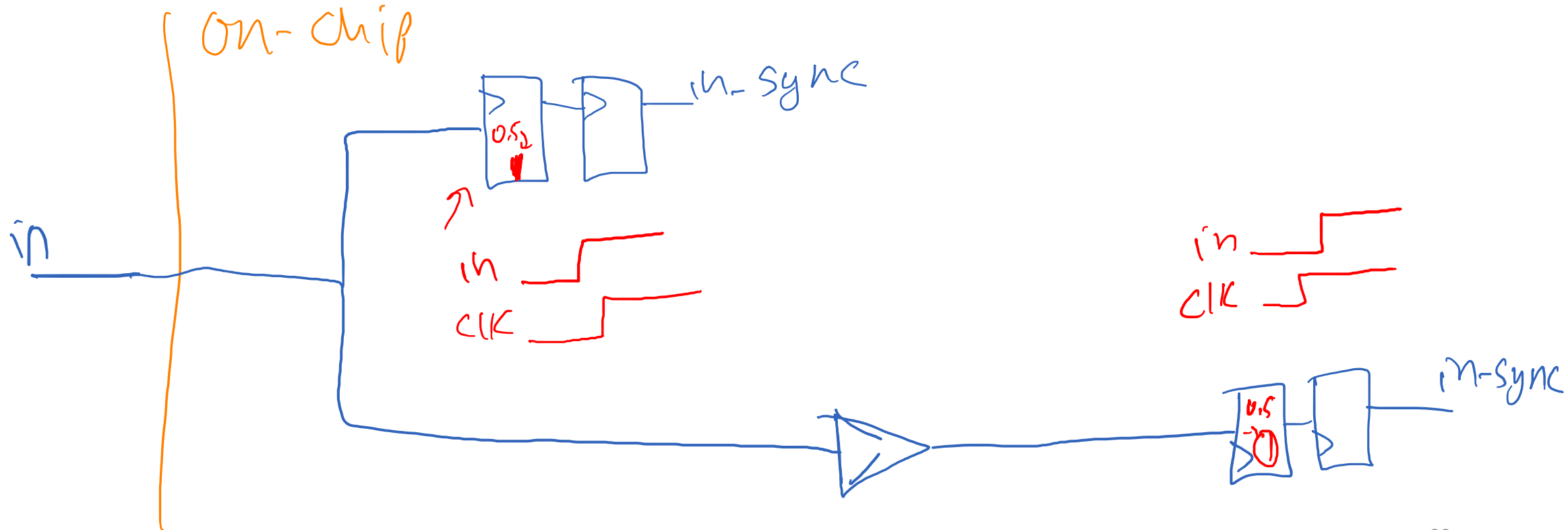
```
set_output_delay -clock [get_clocks clk] -max -add_delay 400 [all_outputs]
```



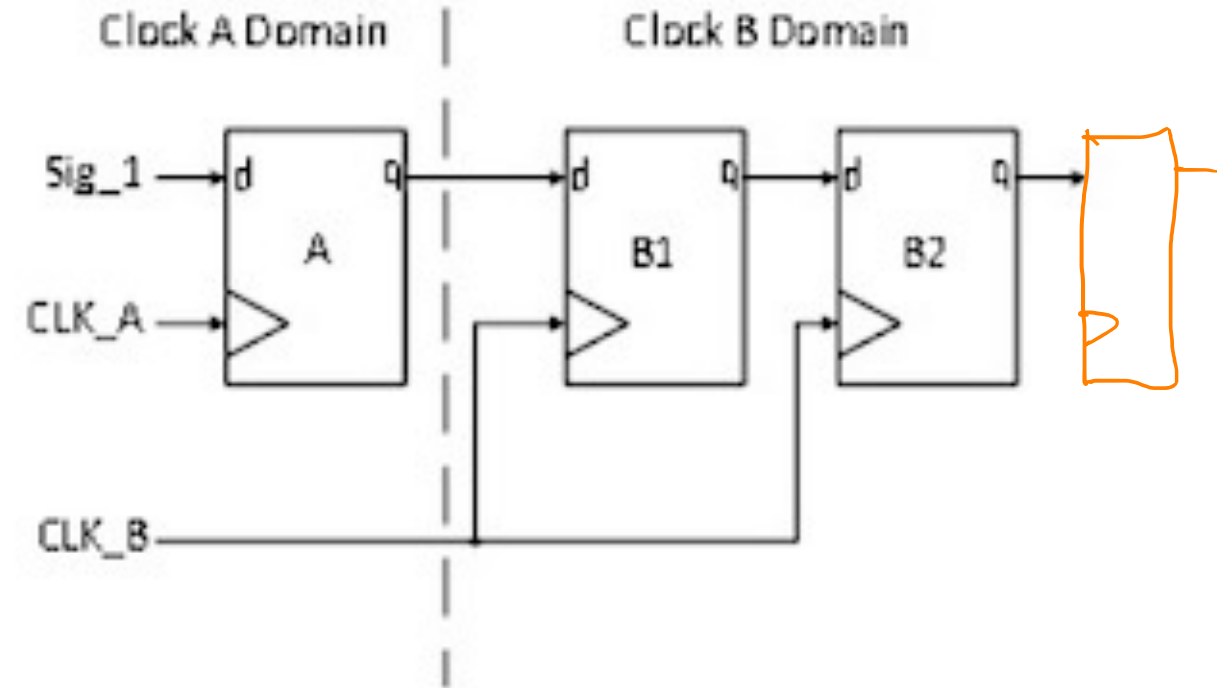
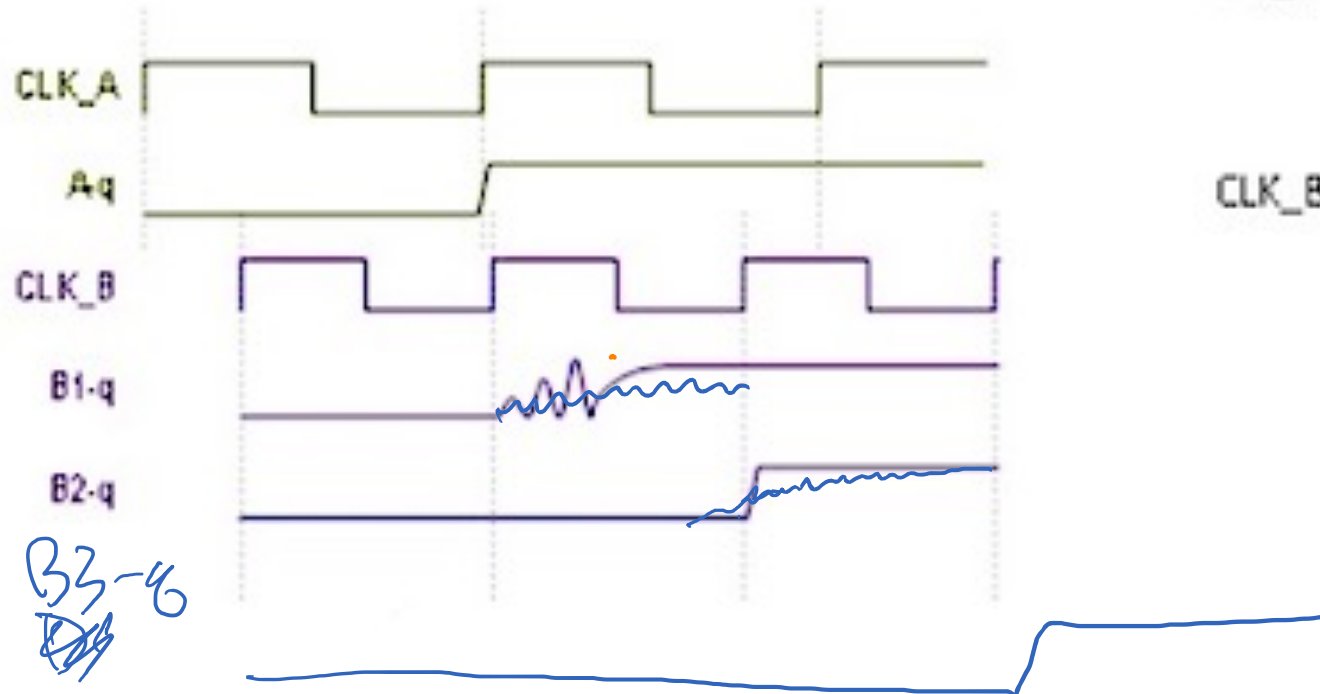
We synchronize the input

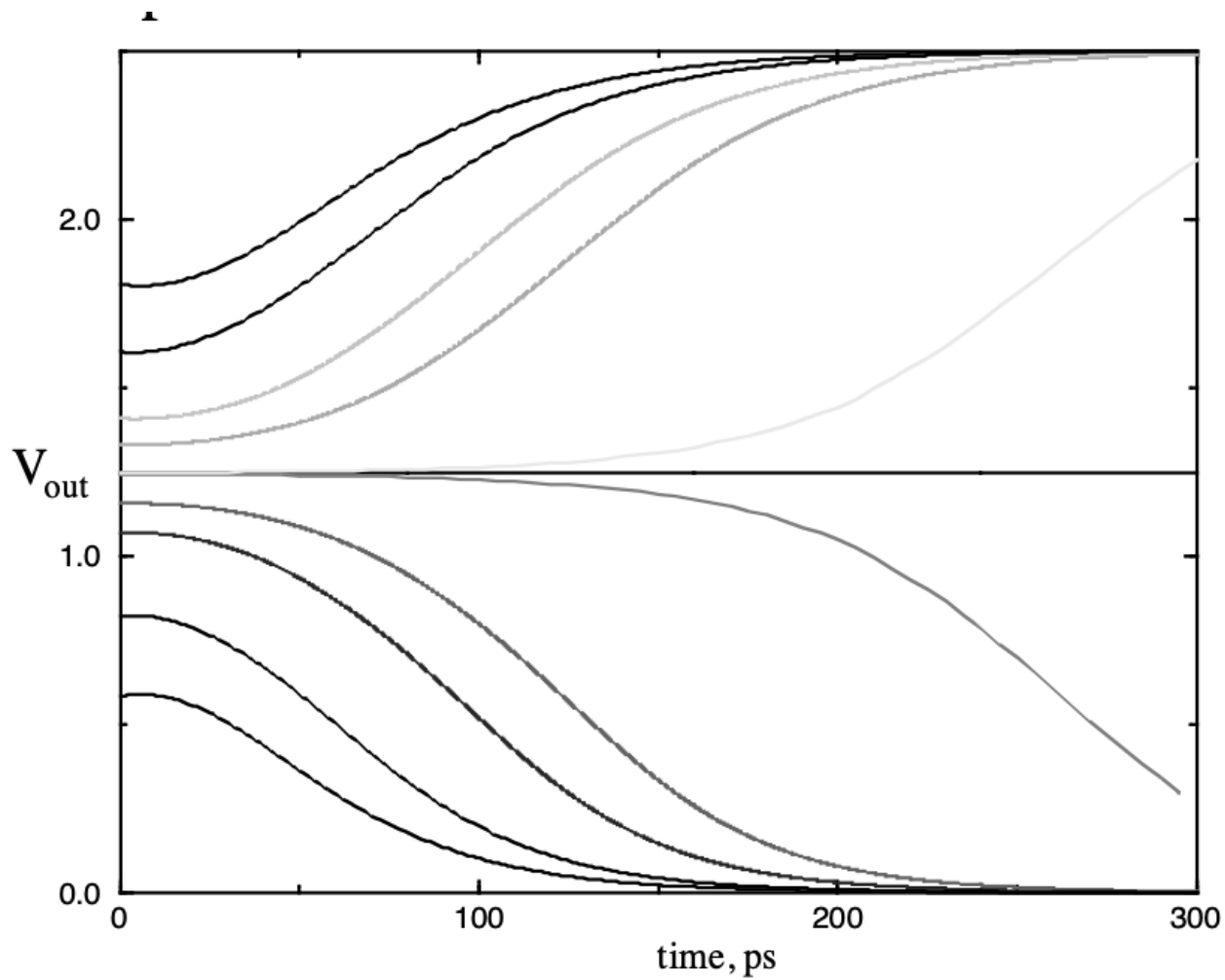
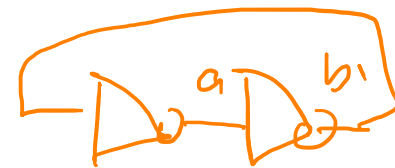
- Metastability
- Multiple path delays

always synchronize (asyn) inputs
do it only 1 time...



We synchronize the input

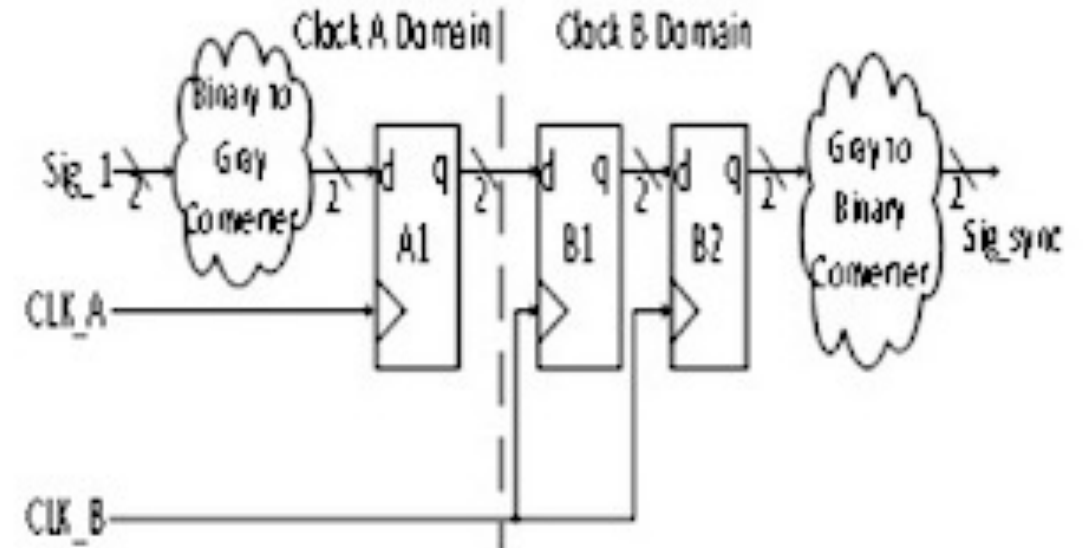
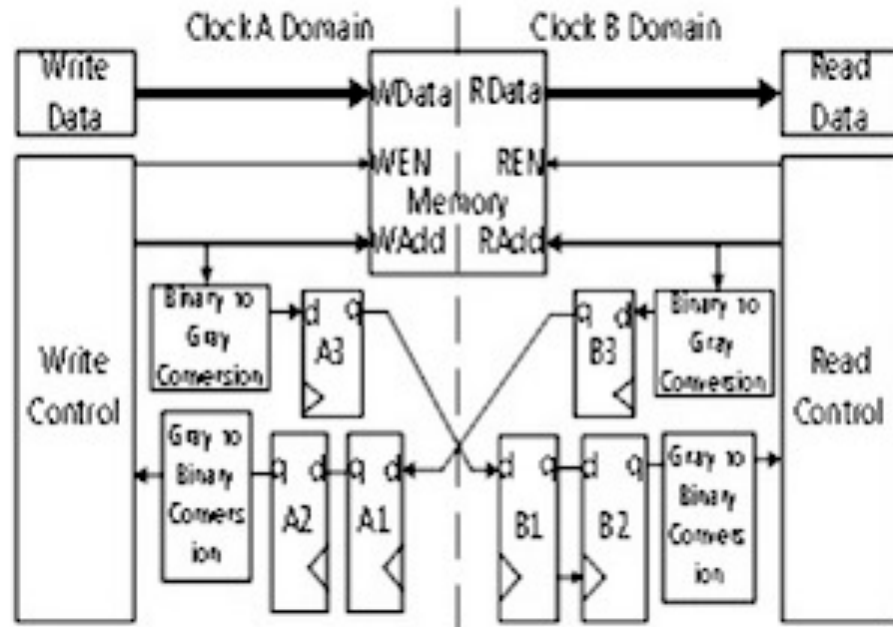




Handwritten notes in orange ink showing the relationship between input values and output values:

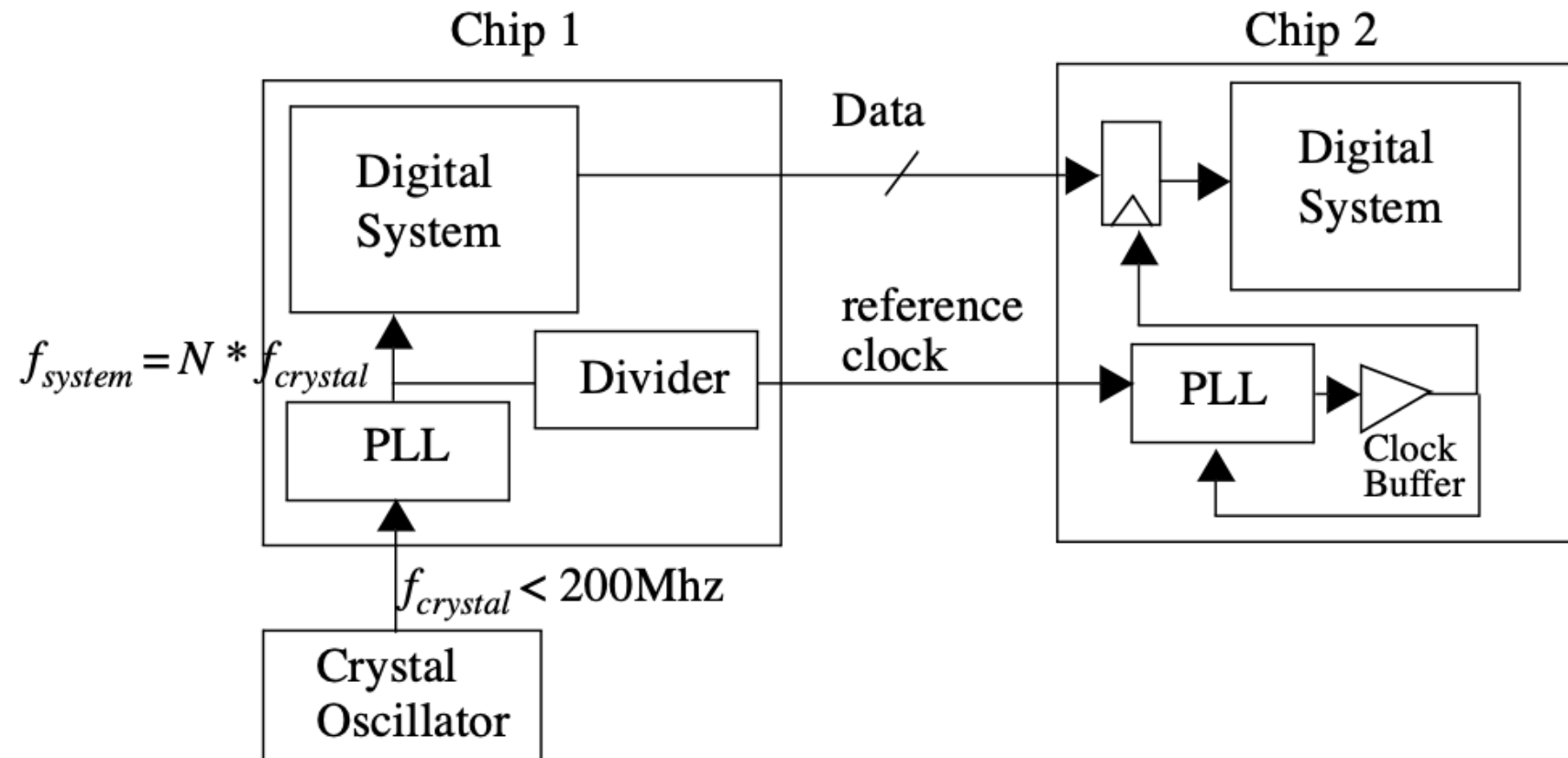
<u>a</u>	<u>b</u>
0.4	0.65
0.3	0.75
0.2	0.85
0.1	0.95
0.0	

Other synchronizer options



00 → 0
 01 → 1
 11 → 2
 10 → 3

We cheat and make them synchronous inputs



What's the main exception here?

```
create_clock -name clk -period 500 [get_ports {clk}]
```

```
set_input_delay -clock [get_clocks clk] -min -add_delay 100 [all_inputs]
```

```
set_input_delay -clock [get_clocks clk] -max -add_delay 400 [all_inputs]
```

```
set_output_delay -clock [get_clocks clk] -min -add_delay 2 [all_outputs]
```

```
set_output_delay -clock [get_clocks clk] -max -add_delay 400 [all_outputs]
```

What's the main exception here? RESET

```
create_clock -name clk -period 500 [get_ports {clk}]
```

```
set_input_delay -clock [get_clocks clk] -min -add_delay 100 [all_inputs]
```

```
set_input_delay -clock [get_clocks clk] -max -add_delay 400 [all_inputs]
```

```
set_output_delay -clock [get_clocks clk] -min -add_delay 2 [all_outputs]
```

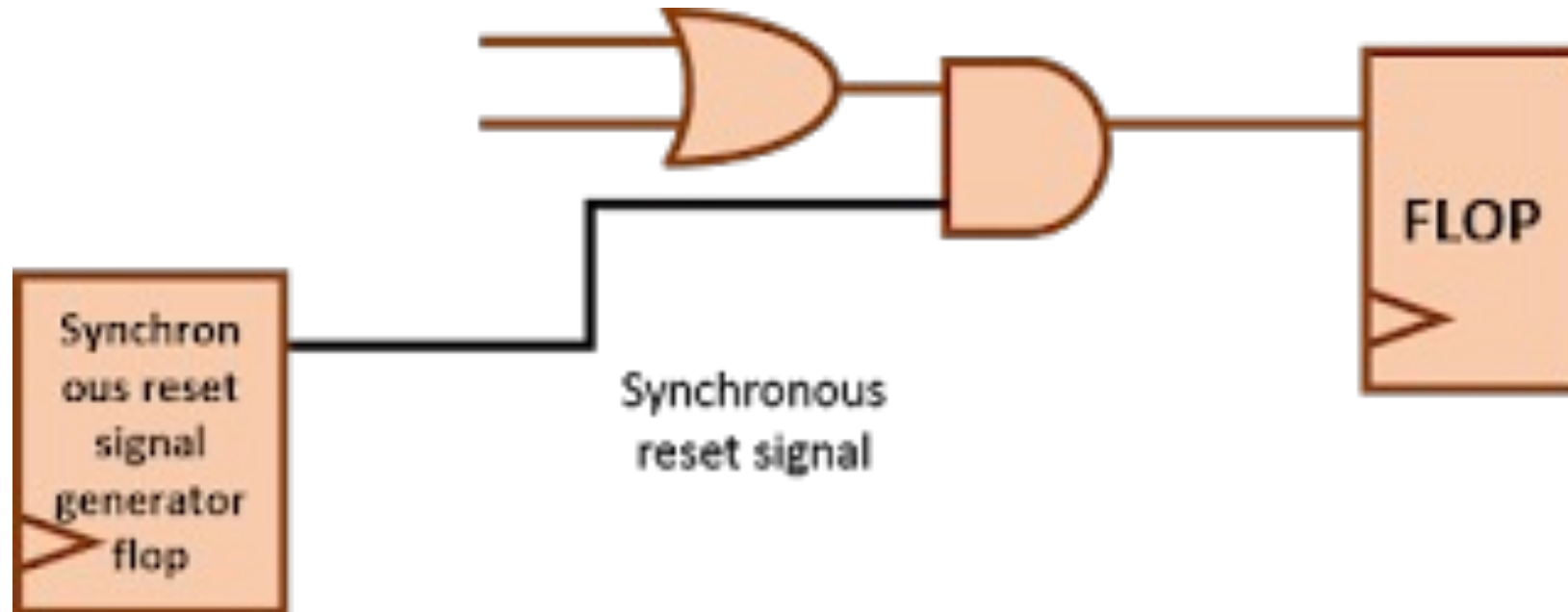
```
set_output_delay -clock [get_clocks clk] -max -add_delay 400 [all_outputs]
```

What makes RESET special?

- A RESET forces all sequential elements (like flip-flops) into a known, stable state, preventing undefined behavior during power-up or operation.
- Two ways to implement reset internally:
 - Asynchronous
 - Synchronous
- Where does reset come from?

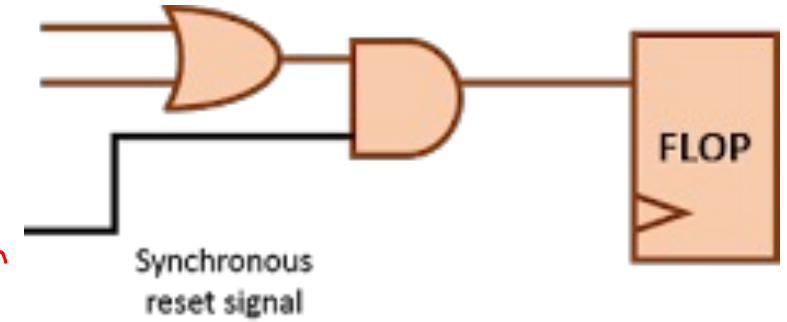
Sync Reset.

- Reset is synchronized to the clock
- Reset just another input
- Usually used for internal reset signals



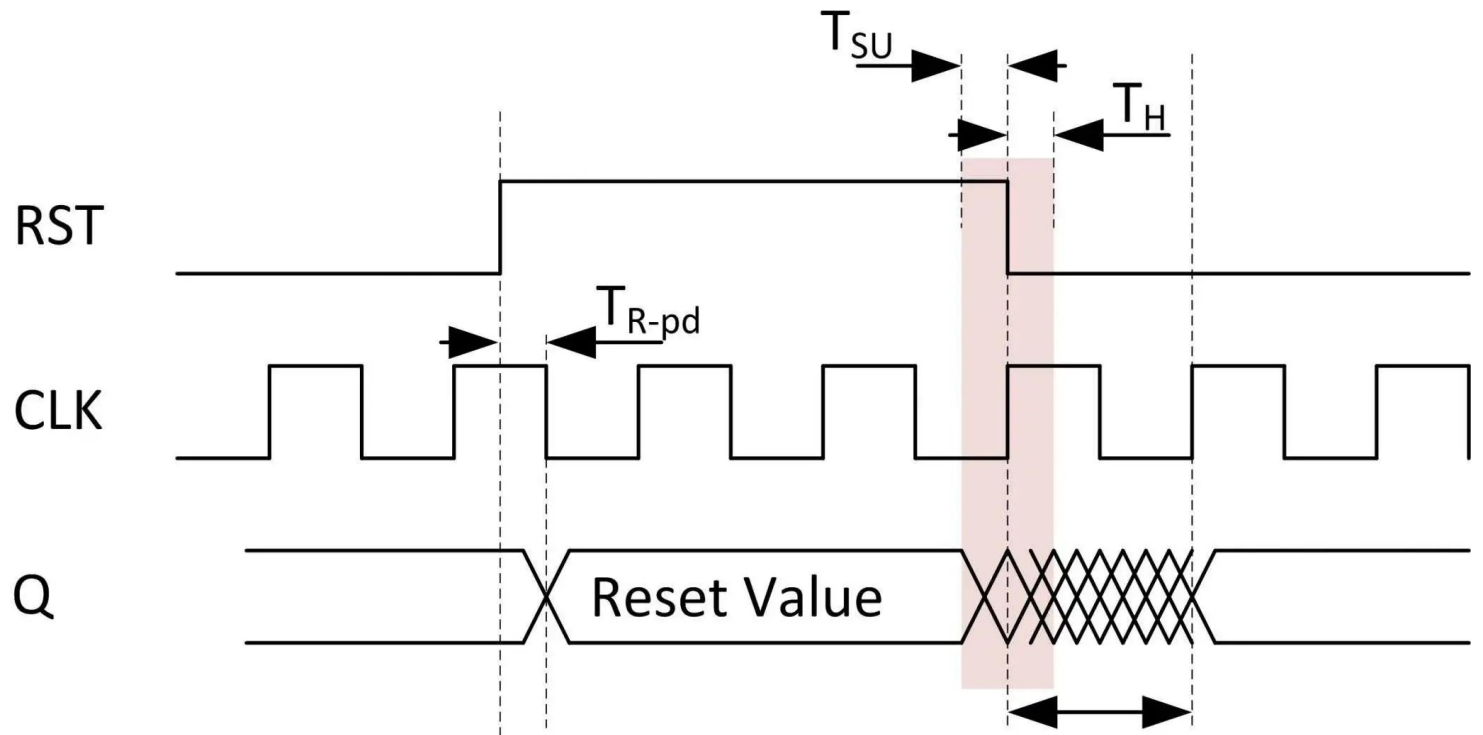
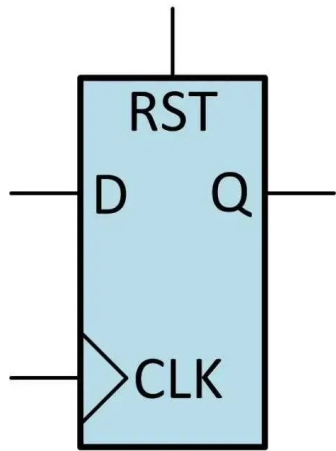
Sync Reset.

```
@always_ff @(posedge clk) begin
    if(rst)
        out q <= 'h0;
    else
        q <= d;
end
```



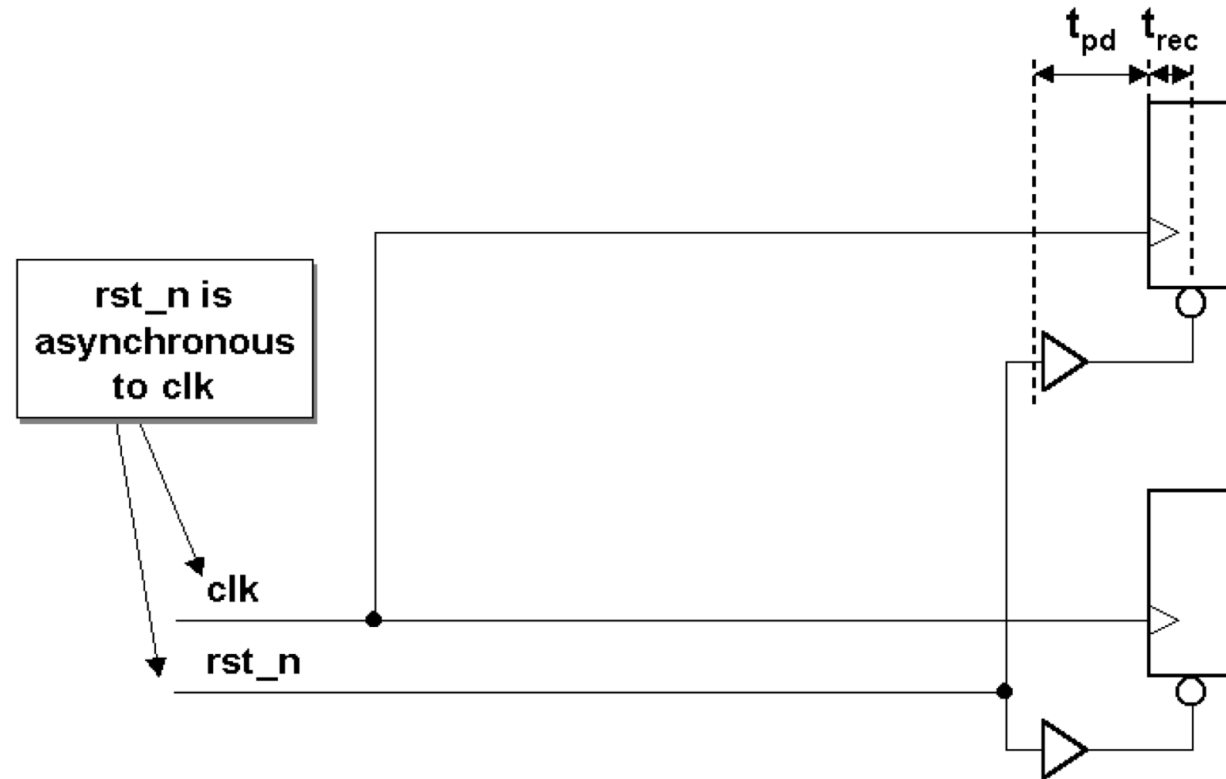
Async Reset.

- Reset is asynchronized to the clock
- Reset special input to flop
- Usually used for external reset signals



(a) A Deterministic
Reset Propagation Delay

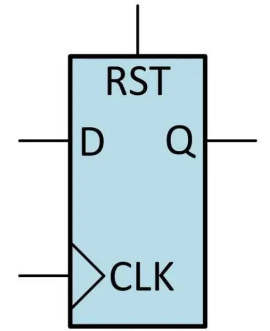
(b) A possibly
metastable value



Async Reset.

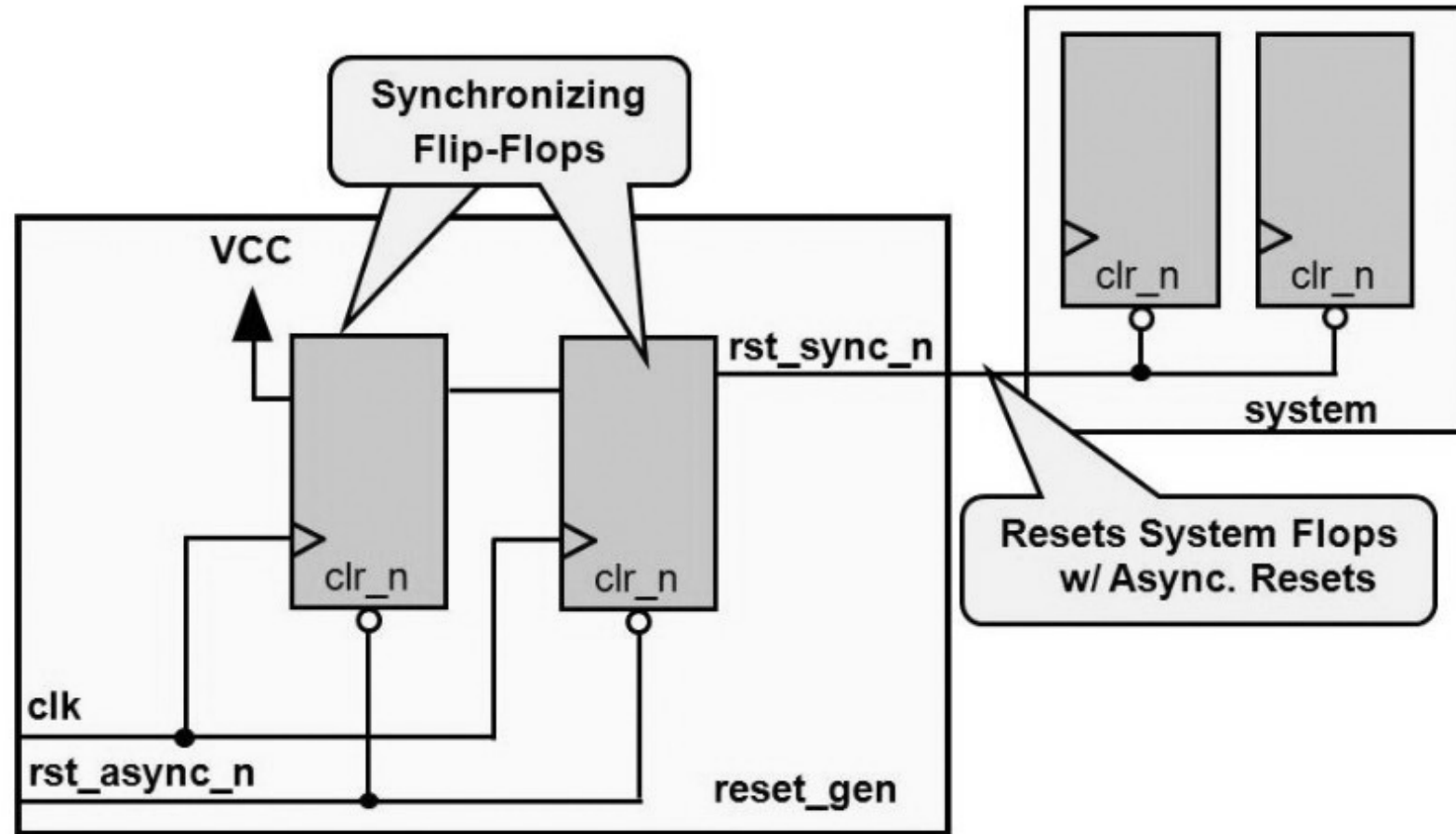
always_ff @(posedge clk, posedge rst)

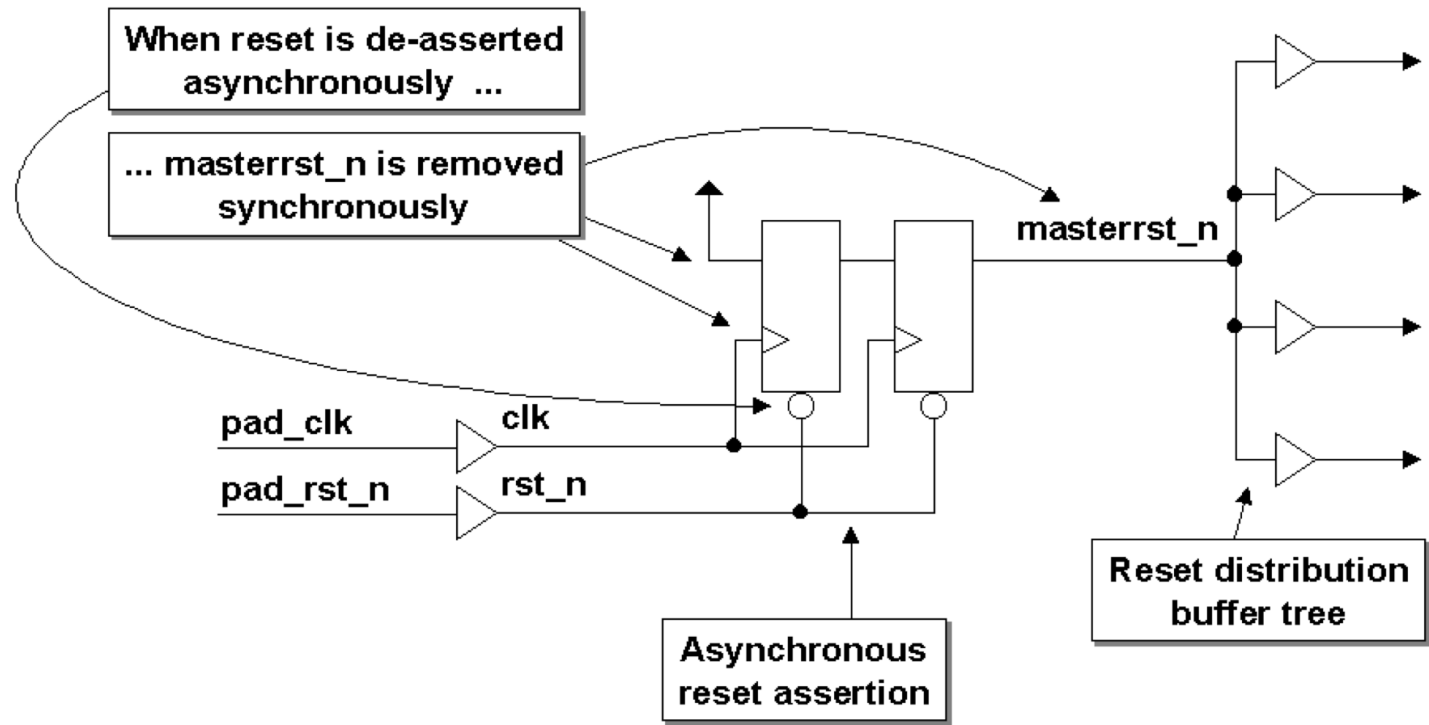
```
if (rst)
    q <= 'h0;
else
    q <= 0;
```

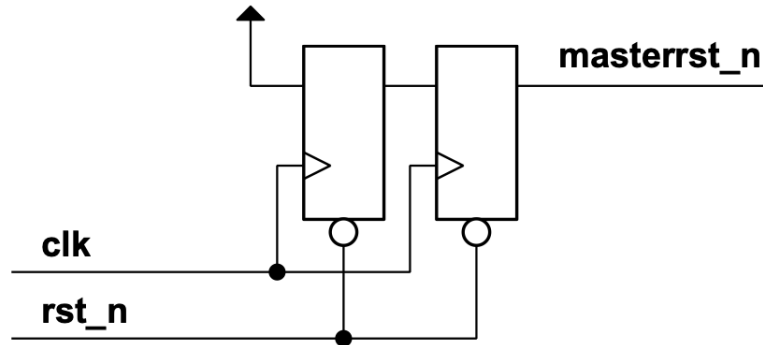


Async Set / Sync Removal Reset

Async Set / Sync Removal Reset







Guideline: EVERY ASIC USING AN ASYNCHRONOUS RESET SHOULD INCLUDE A RESET SYNCHRONIZER CIRCUIT!!

```

module reset_synchronizer (masterrst_n, clk, rst_n);
  output masterrst_n;
  input  clk, rst_n;
  reg    rst_n, rff1;

  always @(posedge clk or negedge rst_n)
    if (!rst_n) {masterrst_n,rff1} <= 2'b0;
    else       {masterrst_n,rff1} <= {rff1,1'b1};
endmodule

```

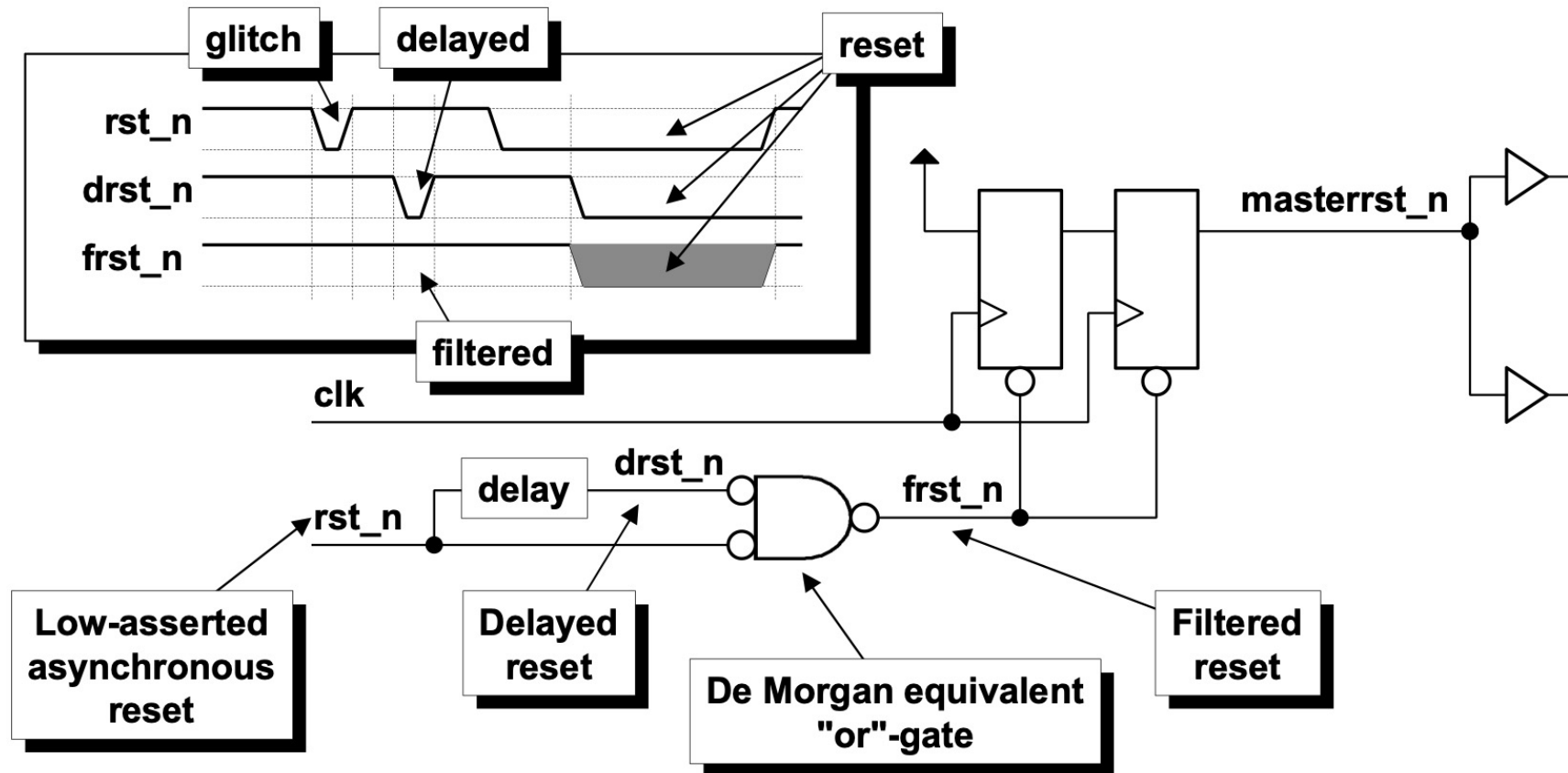
**Asynchronous
reset assertion
(on negedge rst_n)**

**Synchronous
reset removal
(on posedge clk)**

**Concatenation makes for efficient
coding of the reset synchronizer**

Stop

- Glitches on the rst_n input might cause stray resets
- Solution: glitch-filter on the rst_n input



Clock Distribution to minimize skew

- H-Tree
- Fishbone

H-Tree vs. Fish-Bone

